CHAPTER 1: INTRODUCTION

Agriculture plays a vital role in ensuring food security and economic stability. However, farmers often face challenges in making informed decisions about crop selection, fertilizer usage, rainfall forecasting, and yield estimation. Traditional methods rely heavily on experience and intuition, which may not always align with modern agricultural demands. The Crops Census System is designed as a machine learning–based decision-support tool that transforms raw agricultural data into actionable insights. By integrating predictive models with user-friendly interfaces, the system empowers farmers to make smarter, data-driven decisions that improve productivity, optimize resources, and reduce risks.

Agriculture is a critical sector that sustains economies and livelihoods, yet farmers often struggle with uncertainties in crop selection, fertilizer usage, rainfall forecasting, and yield estimation. The Crops Census System is conceptualized as a machine learning–based decision-support tool that abstracts complex agricultural data into simplified, actionable insights. Its theoretical foundation lies in the application of predictive analytics to enhance farming efficiency and sustainability. Traditional farming decisions are often based on experience or limited local knowledge, which may not account for dynamic environmental and soil conditions. This leads to inefficiencies, reduced yields, and resource mismanagement. There is a need for a data-driven system that can process diverse agricultural datasets and provide reliable recommendations to farmers. To predict suitable crops for specific regions and seasons. To recommend crops based on soil nutrient composition and climatic factors. To suggest fertilizers tailored to soil type and crop requirements. To forecast rainfall patterns using historical meteorological data. To estimate crop yield for better planning and resource allocation. The technology used in this project on backend is Python with machine learning libraries (scikit-learn, pandas, NumPy) and frontend is HTML, CSS, Bootstrap 4, JavaScript for user interaction. Integration a web interface that abstracts technical complexity into simple input-output operations for farmers. Additionally, the system relies on structured datasets in crop datasets is like State, district, season, crop. In soil datasets is like Nutrients (N, P, K), pH, moisture. In Weather datasets is like Temperature, humidity, rainfall history and also in Yield datasets is like Area, production, year, crop type. The Crops Census System embodies the abstraction of agricultural knowledge into a machine learning framework. It bridges the gap between raw

data and farmer-friendly insights by Simplifying complex computations into intuitive recommendations. Enhancing decision-making in crop planning and resource management. Promoting sustainable agriculture through optimized fertilizer usage and yield forecasting.

1.1 Problem Statement

Agricultural decisions such as selecting a suitable crop, choosing fertilizer, estimating crop yield, and planning around rainfall depend on several factors, including soil nutrients, weather, location, season, and cultivated area. Farmers may find it difficult to assess all these factors and access relevant information in one place. As a result, decisions based only on limited information or past experience may not make effective use of available agricultural data.

Crop Census addresses this problem by providing a web-based system that brings together crop recommendations and agricultural predictions. Users can enter relevant field or location details and receive data-based results to support their planning. These results are intended as decision-support information and do not replace advice from agricultural experts.

1.2 Objectives of the Study

The main objective of the project is to develop a web-based agricultural decision-support system that makes selected crop-related recommendations and predictions accessible through a single interface.

The specific objectives are to:

- Recommend a crop based on soil nutrient values, temperature, humidity, pH, and rainfall.
- Provide fertilizer recommendations using soil nutrient values, temperature, humidity, soil moisture, soil type, and crop.
- Show crops associated with a selected state, district, and season.
- Estimate crop yield based on location, season, crop, and cultivated area.
- Estimate rainfall for a selected region and month.
- Provide a straightforward interface for entering information and viewing results.
- Allow users to register, log in, and use the website’s agricultural tools.

1.3 Scope of the Study

The project covers the development of a web application that combines five agricultural tools: crop recommendation, fertilizer recommendation, crop prediction by location and season, crop yield prediction, and rainfall prediction. Users provide the inputs requested by each tool, and the system displays a result generated using the project's datasets and Python-based prediction or recommendation scripts.

The application also includes user registration and login functionality backed by a MySQL database. The web interface is implemented in HTML and uses client-side elements for selecting some location and crop-related values.

The project is limited to the data, locations, crops, seasons, and input fields supported by its datasets and implementation. Its outputs are estimates or recommendations; they may not reflect every local condition and should be considered alongside current field observations and qualified agricultural guidance. The system does not directly collect sensor data, control farm equipment, or guarantee crop outcomes.

1.4 Overview of the System

Crop Census is a web-based agricultural information system. Users access the application through a browser and select the tool relevant to their planning needs. Depending on the tool, they enter soil and weather measurements or select a location, season, crop, area, region, or month.

The application presents the forms and processes submissions. It passes the relevant input values to Python scripts, which use the project's agricultural datasets and prediction logic to generate recommendations or estimates. The result is then displayed on the corresponding page. MySQL is used to store user account information, while the agricultural datasets and model-related files support the prediction and recommendation tools.

At a high level, the system consists of:

- User interface: Browser-accessible pages for the home screen, account access, forms, and results.
- Application layer: HTML pages that handle form submissions and connect the interface to the prediction scripts.
- Prediction and recommendation layer: Python scripts and model logic that process the supplied inputs.
- Data layer: Agricultural datasets and model files used by the tools, plus a MySQL database for user accounts.

CHAPTER 2: LITERATURE SURVEY AND REVIEW

2.1 Introduction

Agriculture depends on many interacting factors, including soil characteristics, weather, location, crop type, and season. Advances in data collection and machine learning have made it possible to analyze these factors and provide farmers with information to support decisions such as crop selection, fertilizer planning, and production estimation.

This chapter reviews common data-driven approaches used in agricultural decision-support systems and describes the techniques applied in Crop Census. The project combines classification methods, a regression model, and a historical-average calculation to provide crop and fertilizer recommendations, crop predictions, yield estimates, and rainfall estimates.

2.2 Literature Survey

Agricultural decision-support systems use historical and field data to help users make informed choices. These systems commonly process input variables such as soil nutrients, temperature, humidity, rainfall, geographic area, and crop season. Their results can help organize information that would otherwise require users to consult multiple sources or analyze datasets manually.

Crop recommendation is generally treated as a classification problem. A model learns relationships between soil and climate measurements and crop labels, then predicts a suitable crop for new input values. Decision trees and ensemble methods such as Random Forests are commonly used for classification. Random Forests combine predictions from multiple decision trees, while a single decision tree makes predictions through a sequence of feature-based decisions [1].

Fertilizer recommendation can also be framed as classification. The system uses nutrient levels, soil and crop categories, and other field measurements to predict a fertilizer label. Decision trees are useful for this task because their branching structure can represent relationships between input conditions and output classes. Categorical information, such as soil type and crop, must be converted into a numerical representation before it can be used by many machine-learning algorithms.

Crop prediction by location and season uses categorical information, such as state, district, and season, to predict crop labels. Decision trees can divide the data into groups based on these input values. Measures such as Gini impurity and information gain help determine which split best separates the crop classes [2].

Crop production estimation is a regression problem when the target is a numeric production value. Random Forest regression can model relationships between production and inputs such as location, crop, season, and area. Categorical variables need encoding before they can be supplied to a model.

Rainfall estimation can use statistical summaries of historical observations. A simple baseline is to calculate the mean rainfall for a selected region and month across available years. This approach is straightforward and interpretable, but it does not account for year-to-year trends, changing climate conditions, or short-term weather patterns.

These approaches offer practical ways to turn agricultural data into recommendations or estimates. Their usefulness depends on the quality, coverage, and relevance of the underlying data. Results should therefore be treated as decision-support information rather than guaranteed outcomes.

2.3 Techniques Used in the Project

The Crop Census system uses the following techniques:

Random Forest classification for crop recommendation: The system uses soil nutrient values (nitrogen, phosphorus, and potassium), temperature, humidity, pH, and rainfall as inputs. A Random Forest classifier, configured with 10 trees, predicts a crop category from the recommendation dataset.

Decision Tree classification for fertilizer recommendation: The system uses nutrient values, temperature, humidity, soil moisture, soil type, and crop type to predict a fertilizer category. Soil type and crop type are label-encoded before the model is trained.

Gini-based Decision Tree for crop prediction: State, district, and season are used to predict crop categories. The tree is constructed by selecting splits according to Gini impurity and information gain. For a selected input, the system follows the matching branches to produce crop predictions.

Random Forest regression for production estimation: The system uses state, district, season, crop, and area to estimate a numerical production value. Categorical features are one-hot encoded, and a Random Forest regressor with 100 trees is trained on the available Karnataka production dataset. Although the interface describes the result as crop yield, the model's target variable is production.

Historical monthly average for rainfall estimation: The system selects historical rainfall records for the chosen region and calculates the average for the selected month. This is a historical-average estimate, not a weather forecast.

In the project implementation, the crop recommendation, fertilizer recommendation, and production-estimation scripts train their models when called. The crop-prediction script loads a previously saved decision tree. These techniques provide a practical foundation for the tools, while their predictive performance depends on the datasets and would need to be assessed with appropriate evaluation measures.

2.4 Summary

This chapter reviewed common data-driven methods for agricultural decision support and described how they are used in Crop Census. The system applies Random Forest classification for crop recommendation, a Decision Tree for fertilizer recommendation, a Gini-based Decision Tree for crop prediction by location and season, and Random Forest regression for production estimation. Rainfall is estimated using historical monthly averages. Together, these techniques support the project's goal of presenting useful agricultural guidance through a single web application.

CHAPTER 3: EXISTING AND PROPOSED SYSTEM

3.1 Existing System and Architecture

In a conventional approach, farmers make crop and fertilizer decisions using personal experience, advice from local sources, and information collected from separate agricultural services. Details about soil nutrients, weather, crop seasons, and regional production may be difficult to bring together, particularly when they are stored in different formats or require manual interpretation.

This can make it challenging to compare options or use historical information consistently. Crop selection and fertilizer planning may not fully account for the field's measured conditions, while production and rainfall estimates may require users to find and analyze relevant data themselves.

The existing approach is not necessarily the same for every farmer or region. In this project, it refers broadly to decision-making without a single integrated application that provides the agricultural tools described below.

3.2 Proposed System

The proposed system, Crop Census, is a web-based agricultural decision-support application. It brings several crop-related tools together so that users can submit relevant field or location information and view a recommendation or estimate.

The system provides:

Crop recommendation: Predicts a suitable crop using nitrogen, phosphorus, potassium, temperature, humidity, pH, and rainfall inputs.

Fertilizer recommendation: Predicts a fertilizer category using nutrient values, temperature, humidity, soil moisture, soil type, and crop type.

Crop prediction: Displays crop predictions for a selected state, district, and season.

Production estimation: Estimates production using state, district, season, crop, and area. The underlying model predicts production, although the page describes the result as yield.

Rainfall estimation: Calculates a historical monthly average for a selected region.

The application uses HTML for its web pages and form processing, Python scripts for recommendations and predictions, agricultural datasets and model files for the underlying data, and MySQL for user account information. Users can register or log in and access the available tools through the website.

The system's outputs are data-based estimates and recommendations. Their usefulness depends on the coverage and quality of the datasets and should be considered alongside local conditions and appropriate agricultural advice.

3.3 Advantages of the Proposed System

Centralized access: Provides several agricultural tools through one web application.

Structured input: Guides users to provide the information needed for each recommendation or estimate.

Data-supported guidance: Uses agricultural datasets and machine-learning techniques to generate results.

Location and season awareness: Includes regional and seasonal inputs in relevant crop and production tools.

Convenient results: Displays outputs through the same interface used to enter information.

Account support: Includes user registration and login functionality.

Potential to support planning: Helps users consider crop suitability, fertilizer options, production estimates, and historical rainfall when making decisions.

CHAPTER 4: SYSTEM ANALYSIS

4.1 Functional Requirements

Functional requirements describe the services the system must provide to its users.

User registration: The system shall allow a user to create an account by entering the required registration details.

User login and logout: The system shall allow registered users to log in and end their session.

Crop recommendation: The system shall accept nitrogen, phosphorus, potassium, temperature, humidity, soil pH, and rainfall values, then display a recommended crop.

Fertilizer recommendation: The system shall accept nutrient values, temperature, humidity, soil moisture, soil type, and crop type, then display a fertilizer recommendation.

Crop prediction: The system shall accept a state, district, and season, then display crop predictions for the selected location and season.

Production estimation: The system shall accept a state, district, season, crop, and area, then display an estimated production value.

Rainfall estimation: The system shall accept a region and month, then display an estimate based on historical rainfall records.

Input validation: The system shall require the inputs needed by each tool and limit selections to the options provided by its form.

Result display: The system shall show the output for the submitted inputs on the relevant tool page.

Account data storage: The system shall store user account information in the MySQL database.

4.2 Non-Functional Requirements

Non-functional requirements describe the quality and operating characteristics of the system.

Usability: Forms and navigation should be understandable to users with basic computer and web-browsing skills.

Performance: The system should return results within a reasonable time, depending on server capacity, dataset size, and model execution.

Reliability: The application should process valid inputs consistently and display results from the available datasets and models.

Security: Passwords should be stored as hashes, and database access should use appropriate controls. User input passed between the HTML application and Python scripts should be handled safely.

Maintainability: Pages, prediction scripts, datasets, and account storage should remain organized so that individual components can be updated.

Compatibility: The web interface should work in current browsers that support standard HTML, CSS, and JavaScript.

Data limitations: Recommendations and estimates depend on the quality and coverage of the available data and must not be presented as guaranteed outcomes.

Availability: The application requires its web server, database, Python runtime, and relevant data files to be available for the corresponding features to work.

4.3 Feasibility Study

Technical Feasibility

The project is technically feasible using the technologies already used in the application: HTML for the web interface and form processing, Python for prediction and recommendation scripts, MySQL for account information, and agricultural datasets for model inputs. It can run on a standard web-server environment, provided that HTML can invoke the required Python scripts and the Python libraries and data files are installed and accessible.

Economic Feasibility

The system can be developed and operated using widely available software, including HTML, Python, MySQL, and commonly used machine-learning libraries. This can keep software licensing costs low. Hardware, hosting, development, maintenance, and data preparation may still involve costs, depending on how the application is deployed.

Operational Feasibility

The application presents its tools through web forms, allowing users to submit field or location information without directly interacting with the datasets or model code. Users need basic web-browsing skills. The usefulness of the results depends on entering appropriate values and on the relevance of the data to the user’s local conditions.

Schedule Feasibility

The project is suitable for incremental development. The interface, account functionality, agricultural datasets, and prediction scripts can be developed and tested as separate components, then integrated into the web application. The time required for completion depends on the scope of testing, data preparation, and deployment.

4.4 Hardware and Software Requirements

Server or development machine:

64-bit processor, dual-core or better

4 GB RAM or more

Approximately 2 GB of free storage for the application, datasets, and software; additional space may be needed for backups or expanded datasets

Internet or local network access for browser-based use

User device:

Desktop, laptop, tablet, or smartphone capable of running a modern web browser

Network access to the application

Software Requirements

Windows or Linux operating system

Apache or another compatible web server

MySQL PDO driver enabled

MySQL database server

Python runtime compatible with the project's scripts

Python libraries used by the scripts, including pandas, NumPy, and scikit-learn

Modern web browser with JavaScript enabled

Project datasets and model files installed in their expected locations

The application and Python scripts must be configured so that the web server can execute the scripts and access the required datasets and model files.

CHAPTER 5: SYSTEM DESIGN

5.1 Proposed System Architecture

Crop Census consists of the following main modules:

User Account Module

Supports account registration, sign-in, and sign-out. Account details are stored in the MySQL users table. Users can access the agricultural tools whether or not they are signed in.

Crop Recommendation Module

Accepts soil nutrient values, temperature, humidity, pH, and rainfall. A Random Forest classifier uses these inputs to recommend a crop.

Fertilizer Recommendation Module

Accepts soil nutrient values, temperature, humidity, soil moisture, soil type, and crop type. A Decision Tree classifier predicts a fertilizer recommendation.

Crop Prediction Module

Accepts state, district, and season, then uses a saved decision-tree model to display crop predictions.

Production Estimation Module

Accepts state, district, season, crop, and area. A Random Forest regressor estimates production. Although the page calls this a yield prediction, the model's target variable is production.

Rainfall Estimation Module

Accepts a region and month, then calculates the average historical rainfall for that month and region.

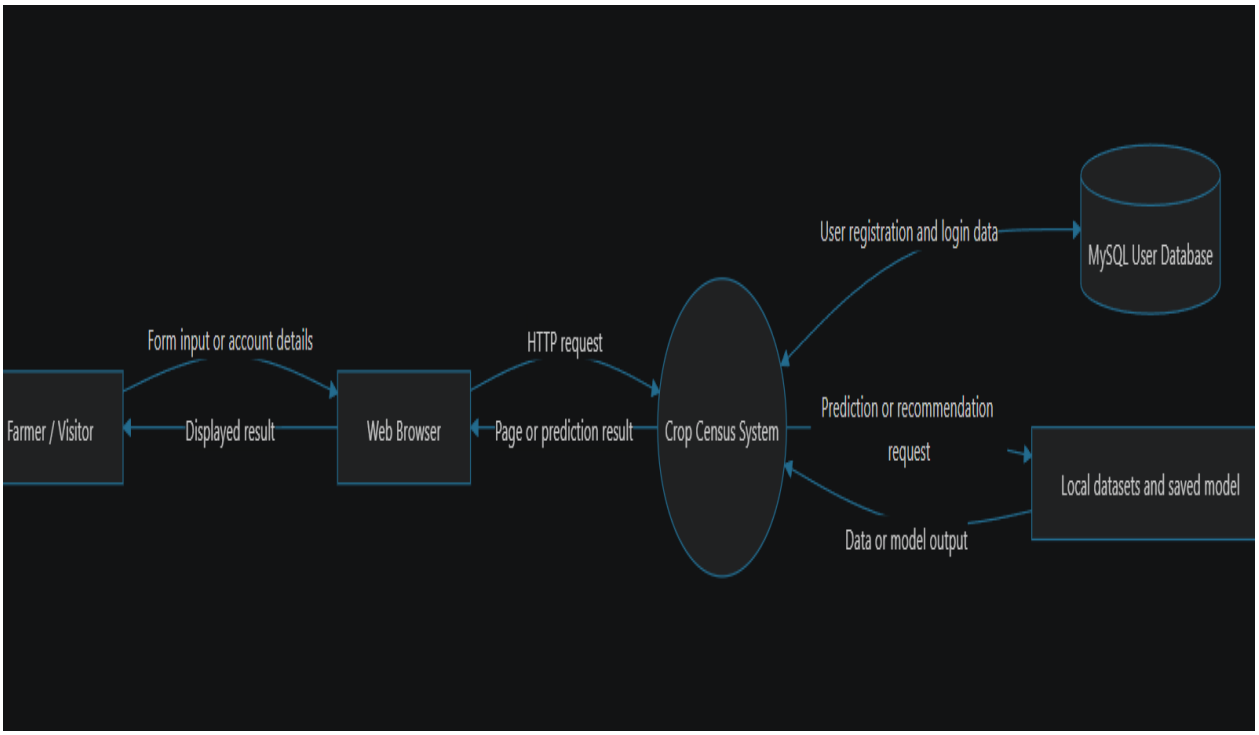
Data and Model Module

Provides the agricultural CSV datasets and saved model file used by the prediction and recommendation scripts.

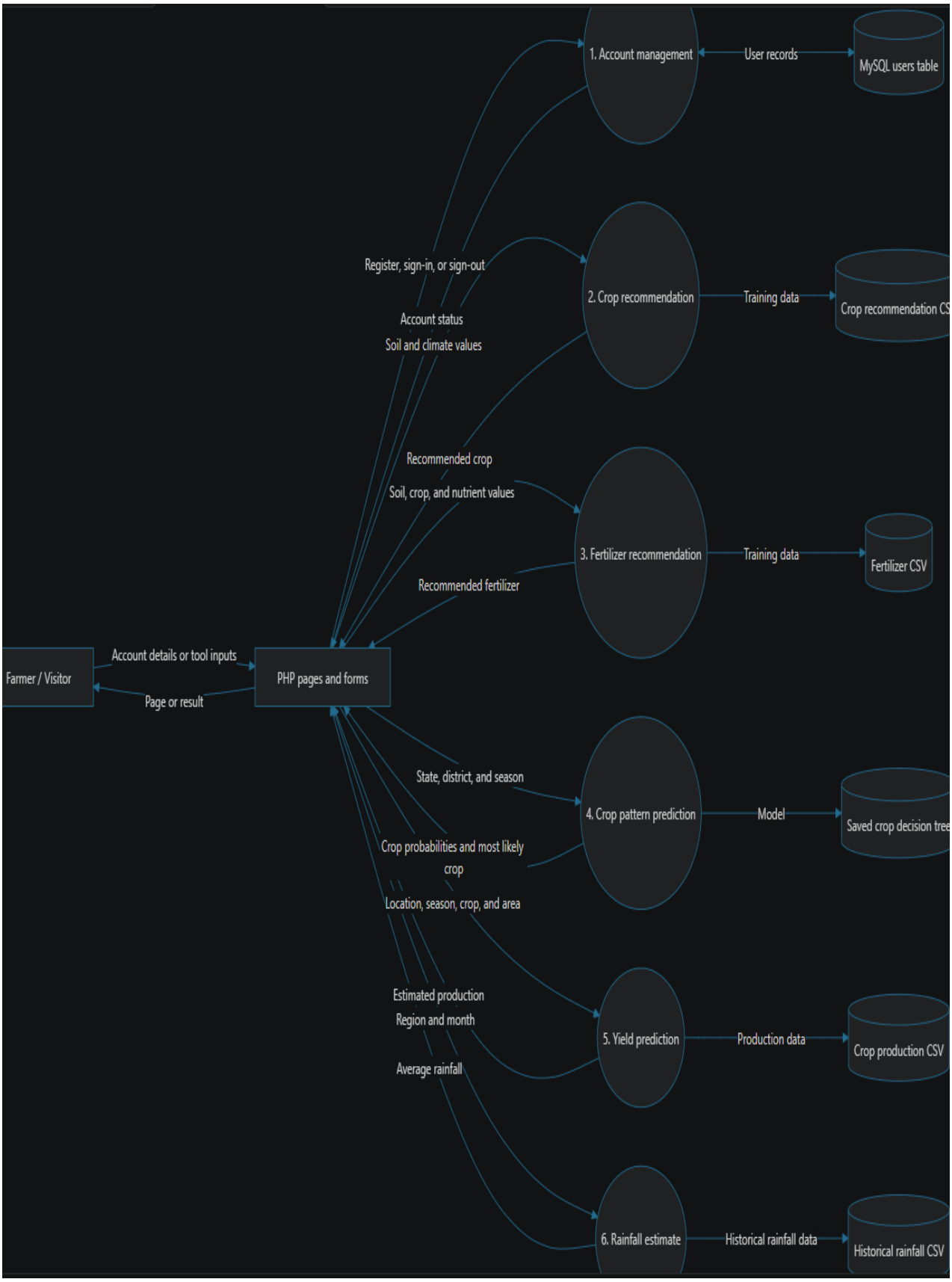
5.2 Data Flow Diagram (DFD)

5.2.1 Level 0 DFD (Context diagram)

This diagram shows the system as a single process and its interactions with users and stored data.

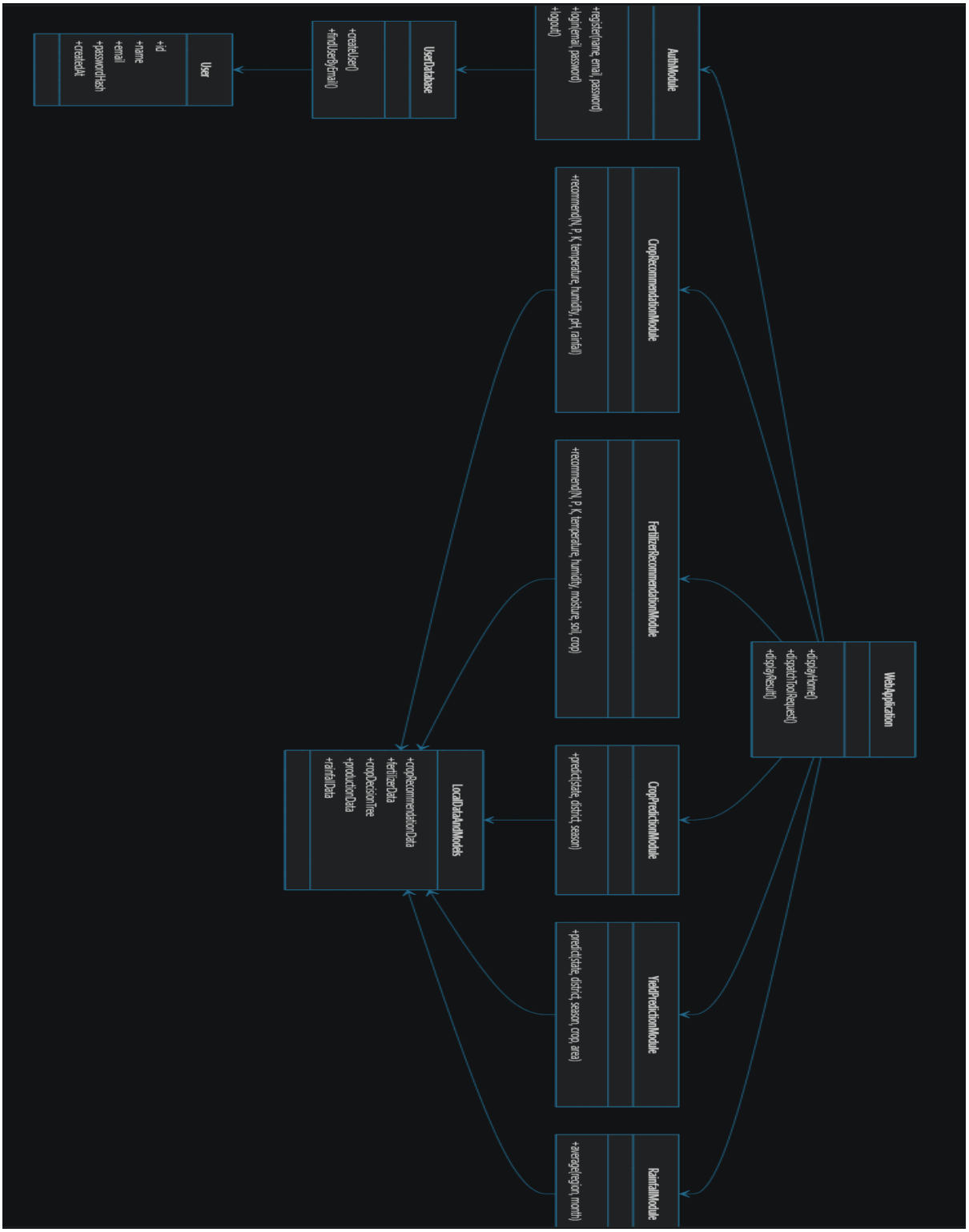


5.2.2 Level 1 DFD



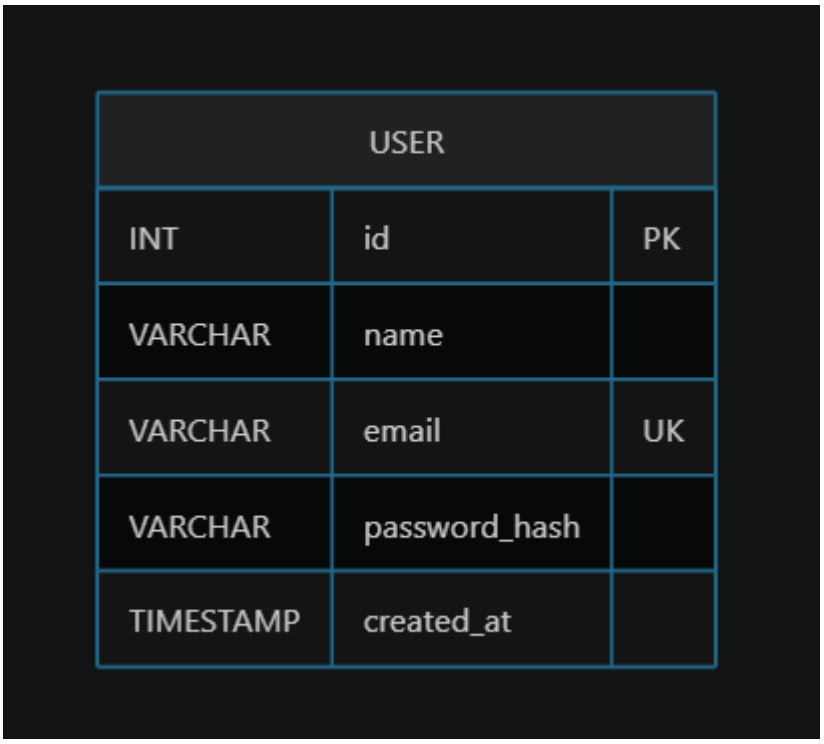
5.3 Class Diagram

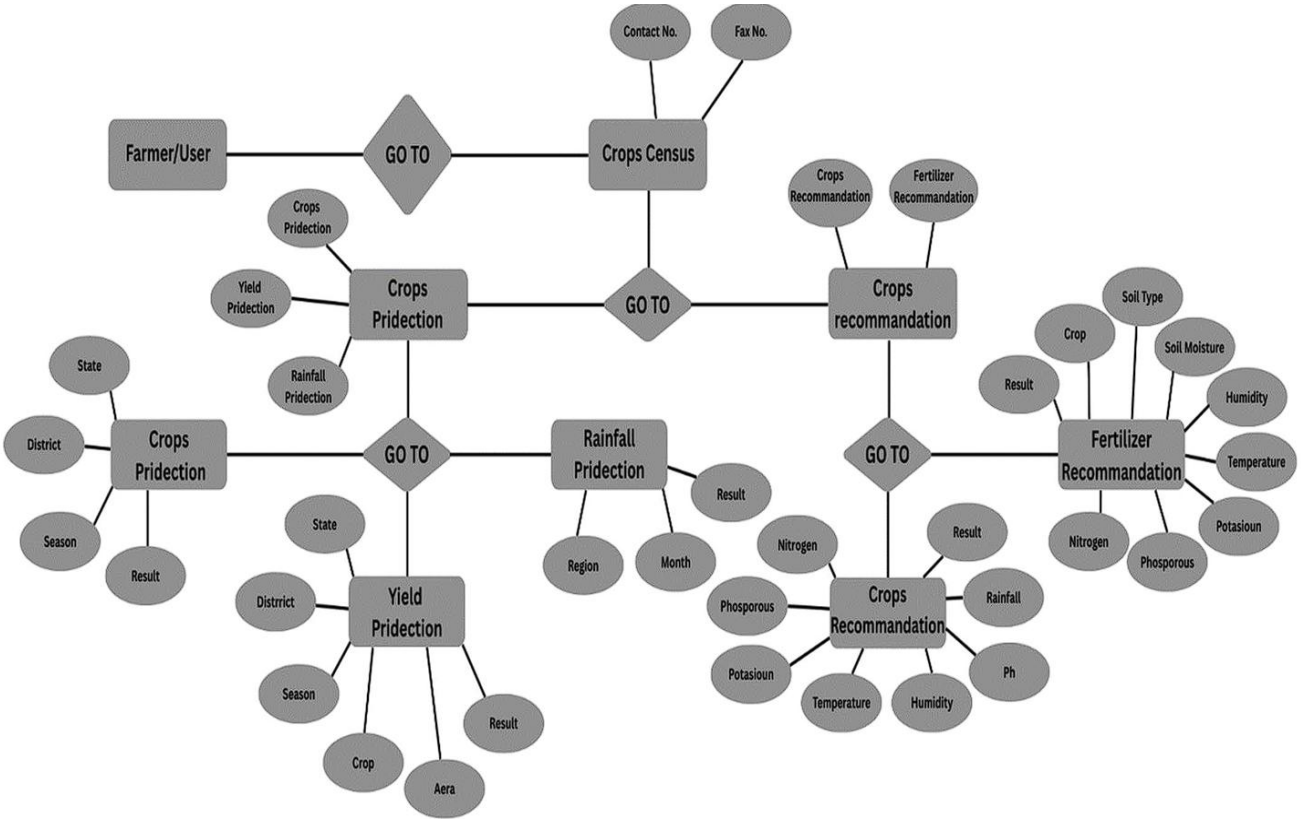
The application is implemented primarily as page scripts and the Python prediction logic as scripts, rather than as classes. This conceptual class diagram groups those existing responsibilities into logical components; it does not imply that these classes are present in the source code.



5.4 Entity Relationship Diagram

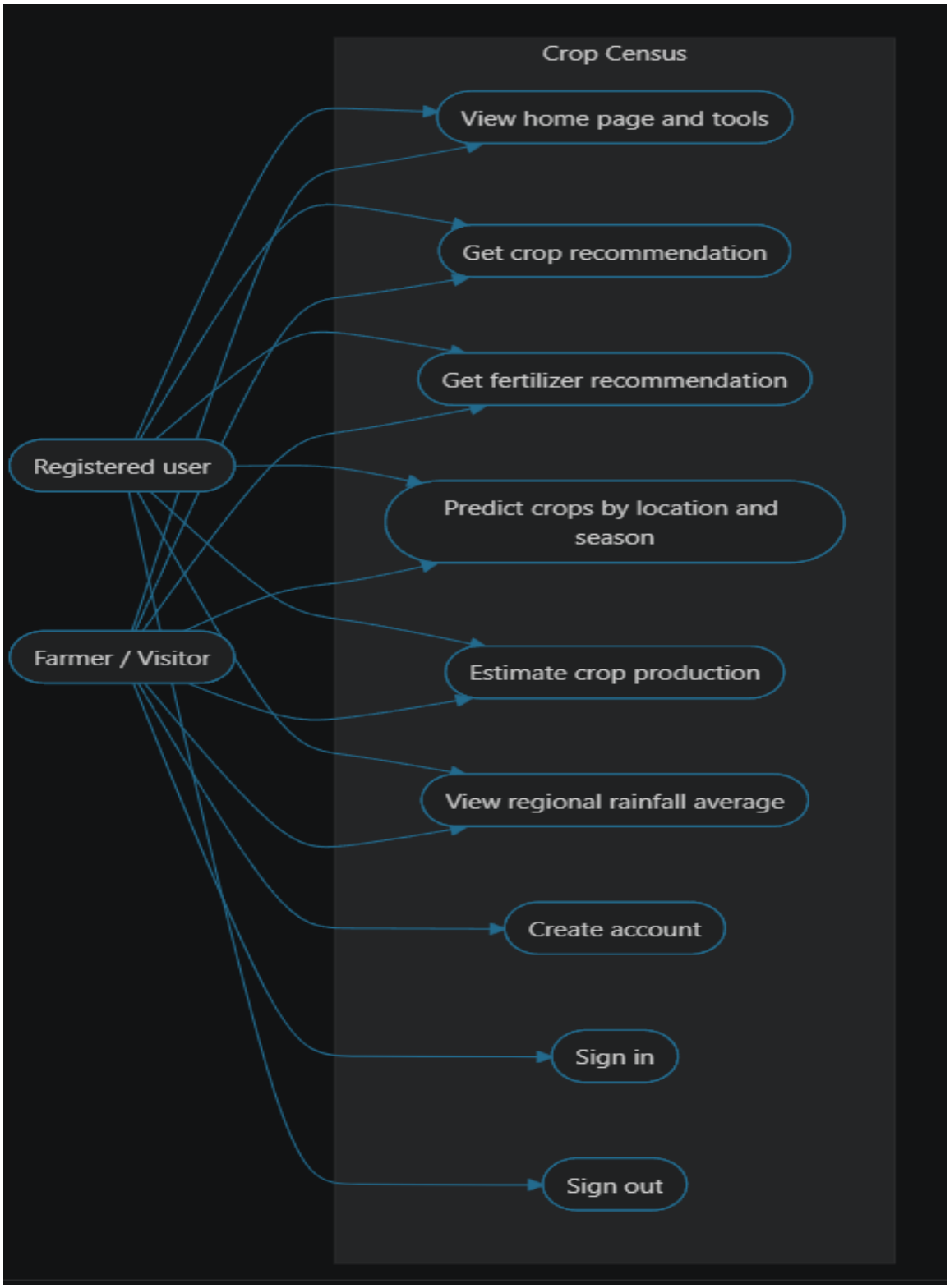
The current database contains one entity: USER. It has no relationships to prediction records because prediction results are not saved in the database





5.5 Use Case Diagram

The tools can be used by a farmer or visitor without signing in. Account use cases are available separately to users who register.



CHAPTER 6: PROJECT PLANNING

6.1 Introduction

Crop Census is implemented as a web application supported by Python scripts for agricultural recommendations and predictions. It displays the forms, processes user input, and presents results. The Python scripts load local datasets or a saved model, produce a recommendation or estimate, and return the result to the page. MySQL stores registered user accounts.

The implementation covers five agricultural tools: crop recommendation, fertilizer recommendation, crop pattern prediction, yield prediction, and rainfall estimation. Account functionality includes registration, sign-in, and sign-out. The main implementation files include config, the individual tool pages, and the scripts under ML.

6.2 Coding Snippets

The following snippets illustrate key implementation patterns in the project.

Database connection and user table initialization

The application uses PDO to connect to MySQL and creates the users table if it does not already exist. Database settings can be supplied through environment variables.

CODE:

```
$databaseHost = getenv('CROP_DB_HOST') ?: '127.0.0.1';
$databaseName = getenv('CROP_DB_NAME') ?: 'crop_census';
$databaseUser = getenv('CROP_DB_USER') ?: 'root';
$databasePassword = getenv('CROP_DB_PASSWORD') ?: '';

$pdo = new PDO(
    'mysql:host=' . $databaseHost . ';dbname=' . $databaseName . ';charset=utf8mb4',
    $databaseUser,
    $databasePassword,
    array(
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
PDO::ATTR_EMULATE_PREPARES => false
```

```
 )
);
```

Account registration:

Registration validates submitted account details, hashes the password, and inserts the new user using a prepared statement.

CODE:

```
if (!$errors) {
    $statement = $pdo->prepare(
        'SELECT id FROM users WHERE email = ?'
    );
    $statement->execute(array($email));

    if ($statement->fetch()) {
        $errors[] = 'An account with that email already exists.';
    } else {
        $statement = $pdo->prepare(
            'INSERT INTO users (name, email, password_hash) VALUES (?, ?, ?)'
        );
        $statement->execute(array(
            $name,
            $email,
            password_hash($password, PASSWORD_DEFAULT)
        ));
    }
}
```

Crop recommendation:

The crop recommendation script loads its dataset, separates features from labels, trains a Random Forest classifier, and predicts a crop from the submitted values.

CODE:

```
dataset = pd.read_csv('ML/crop_recommendation/Crop_recommendation.csv')

X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=0
)

classifier = RandomForestClassifier(
    n_estimators=10,
    criterion='entropy',
    random_state=0
)
classifier.fit(X_train, y_train)

user_input = np.array([[
    n_params, p_params, k_params,
    t_params, h_params, ph_params, r_params
]])
predictions = classifier.predict(user_input)

print(str(predictions[0]))
```

Calling the prediction script:

The crop recommendation page passes the submitted input values to its Python script and displays the returned output.

CODE:

```
$Jsonn = json_encode($n);
$Jsonp = json_encode($p);
$Jsonk = json_encode($k);
$Jsont = json_encode($t);
$Jsonh = json_encode($h);
$Jsonph = json_encode($ph);
$Jsonr = json_encode($r);

$command = escapeshellcmd(
    "python ML/crop_recommendation/recommend.py " .
    "$Jsonn $Jsonp $Jsonk $Jsont $Jsonh $Jsonph $Jsonr"
);
$output = passthru($command);
echo $output;
```

Rainfall Estimate:

The rainfall script filters the historical dataset by region and calculates the selected month's average rainfall.

CODE:

```
def predict_rainfall(state, month):
    state_data = df[df['SUBDIVISION'] == state]
    avg_rainfall = state_data[month].mean()
    return avg_rainfall
```

ML CODE:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import joblib
from collections import Counter
import sys


header = ['State_Name', 'District_Name', 'Season', 'Crop']


class Question:
    def __init__(self,column,value):
        self.column =column
        self.value=value
    def match(self,example):
        val = example[self.column]
        return val == self.value
    def match2(self,example):
        if example == 'True' or example == 'true' or example == '1':
            return True
        else:
            return False
    def __repr__(self):
        return "Is %s %s %s?" %(
            header[self.column],"==",str(self.value))



def class_counts(Data):
    counts= {}
    for row in Data:
```

```
 label =row[-1]
        if label not in counts:
            counts[label] = 0
        counts[label] += 1
    return counts

class Leaf:
    def __init__(self,Data):
        self.predictions = class_counts(Data)



class Decision_Node:
    def __init__(self,question,true_branch,false_branch):
        self.question=question
        self.true_branch = true_branch
        self.false_branch = false_branch




def print_tree(node,spacing=""):
    if isinstance(node,Leaf):
        print(spacing + "Predict",node.predictions)
        return
    print(spacing+str(node.question))
    print(spacing + "--> True:")
    print_tree(node.true_branch,spacing + " ")

    print(spacing + "--> False:")
    print_tree(node.false_branch,spacing + " ")
```

```
def print_leaf(counts):
    total = sum(counts.values())*1.0
    probs = {}
    for lbl in counts.keys():
        probs[lbl] =str(int(counts[lbl]/total * 100)) + "%"
    return probs



def classify(row,node):
    if isinstance(node,Leaf):
        return node.predictions
    if node.question.match(row):
        return classify(row,node.true_branch)
    else:
        return classify(row,node.false_branch)

def get_best_prediction(predictions):
    # predictions is a dict like {'Rice': '70%', 'Wheat': '30%'}
    # Convert percentages to integers and pick the max
    best_crop = max(predictions.items(), key=lambda x: int(x[1].replace('%','')))
    return best_crop[0], best_crop[1]

dt_model_final= joblib.load('ML/crop_prediction/filetest2.pkl')



state =sys.argv[1]
district=sys.argv[2]
season=sys.argv[3]

testing_data = [[state,district,season]]
```

```
for row in testing_data:
    prediction = classify(row, dt_model_final)
    probs = print_leaf(prediction)
    print("Input: State=%s, District=%s, Season=%s" % (row[0], row[1], row[2]))
    print("Predicted Crop Probabilities:", probs)
    Predict_dict = probs.copy()

    # Get best crop
    best_crop, confidence = get_best_prediction(Predict_dict)
    print(f"Most likely crop: {best_crop} ({confidence})")
```

```
import pandas as pd
import numpy as np
import json
import sys

# Get the input parameters as command line arguments
jsonn = sys.argv[1]
jsonp = sys.argv[2]
jsonk = sys.argv[3]
jsont = sys.argv[4]
jsonh = sys.argv[5]
jsonph = sys.argv[6]
jsonr = sys.argv[7]

# Parse the JSON strings into Python objects
n_params = json.loads(jsonn)
p_params = json.loads(jsonp)
k_params = json.loads(jsonk)
t_params = json.loads(jsont)
```

```
h_params = json.loads(jsonh)
ph_params = json.loads(jsonph)
r_params = json.loads(jsonr)

#Read the dataset
dataset = pd.read_csv('ML/crop_recommendation/Crop_recommendation.csv')

#Divide the dataset into features and labels
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, -1].values

#Split the dataset into training and test sets
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, random_state = 0)

#Train the model using the Random Forest Classifier algorithm
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier(n_estimators = 10, criterion = 'entropy', random_state
= 0)
classifier.fit(X_train, y_train)

#Get the user inputs and store them in a numpy array
#user_input = np.array([[90,42,43,21,82,6.5,203]]) - ans is rice.
user_input                                                                    =
np.array([[n_params,p_params,k_params,t_params,h_params,ph_params,r_params]])

#Make predictions using the trained model
predictions = classifier.predict(user_input)

#Print the predicted crop
print(str(predictions[0]))
```

6.3 Testing and Types

Testing should verify the account workflows, the input forms, communication between HTML and Python, and the outputs of the prediction tools. The repository does not include a dedicated automated test suite, so the cases below are a manual testing plan rather than results from an executed test run.

Types of testing

Unit testing: Check individual operations, such as registration validation, password verification, and rainfall average calculation.

Integration testing: Check that pages pass valid form values to Python scripts and correctly display the returned output.

System testing: Check the complete workflow in a browser, including navigation, account pages, prediction forms, and results.

Validation testing: Check that required fields and supported selections are provided before a request is processed.

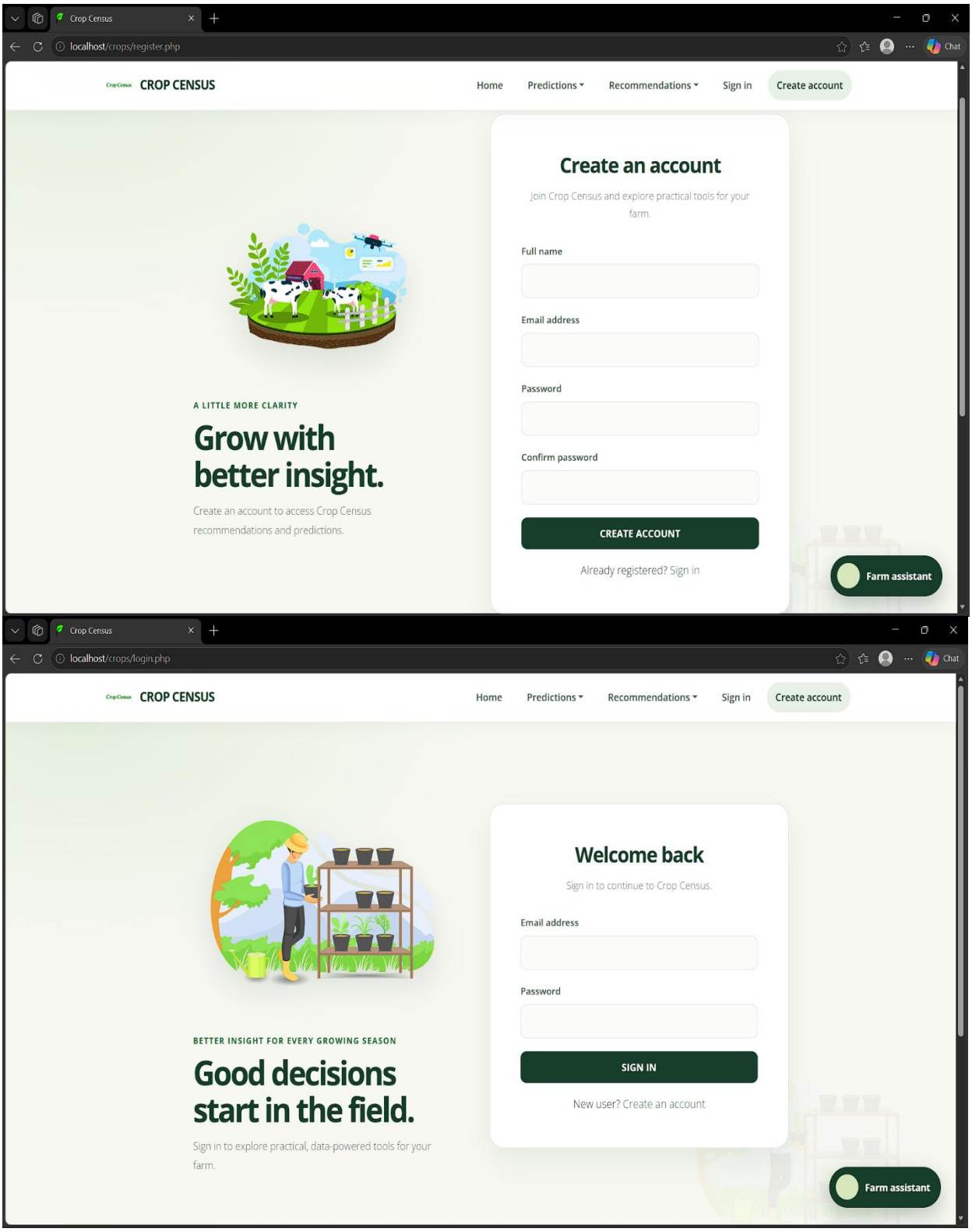
Acceptance testing: Confirm that the tools provide the expected functionality from a farmer’s perspective.

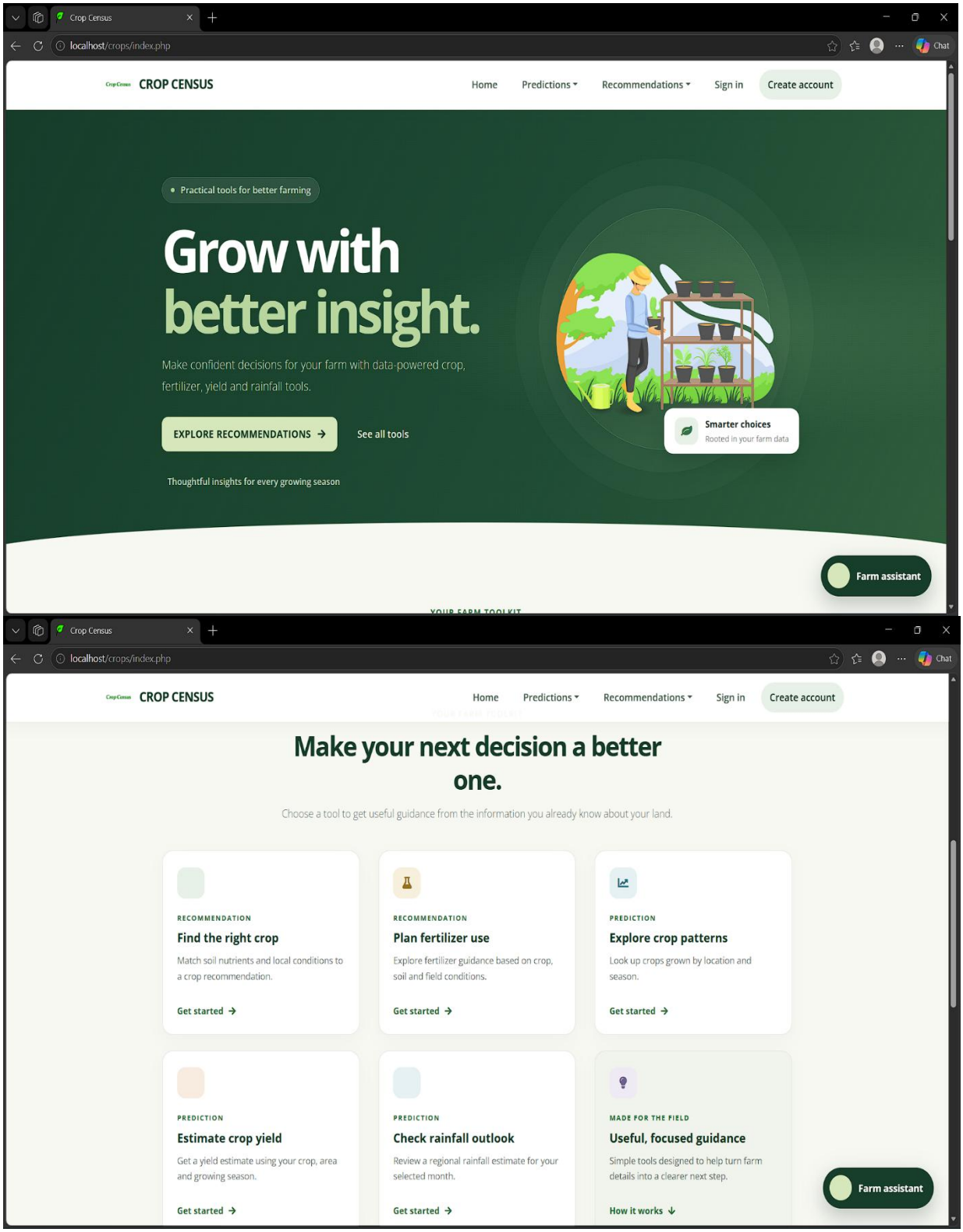
Test ID	Module	Test input or action	Expected result
T01	Registration	Submit a valid name, unused email, and matching passwords	Account is created and the user is directed to sign-in.
T02	Registration	Submit an invalid email or passwords that do not match	Validation errors are displayed and no account is created.
T03	Registration	Register using an email address already in the database	A duplicate-account message is displayed.
T04	Sign-in	Submit a registered email and correct password	User is signed in and redirected to the home page.
T05	Sign-in	Submit an incorrect password	An authentication error is displayed.
T06	Crop recommendation	Enter all seven required soil and climate values	A recommended crop is displayed.
T07	Fertilizer recommendation	Enter valid nutrient, soil, crop, and climate values	A fertilizer recommendation is displayed.
T08	Crop prediction	Select a supported state, district, and season	Crop probabilities and the most likely crop are displayed.
T09	Yield prediction	Select a supported location, season, crop, and area	A predicted production value is displayed.
T10	Rainfall estimate	Select a region and month present in the dataset	The historical average rainfall for that selection is displayed.
T11	Input validation	Submit a tool form with a required field missing	The browser prevents submission or the form reports the missing input.
T12	Sign-out	Sign out while logged in	The session is cleared and the user is redirected to sign-in.

Test status:

These test cases describe expected behavior and should be executed in the project's configured MySQL, and Python environment. Actual test results should be recorded after execution; they are not reported here as passed.

CHAPTER 7: SCREENSHORT

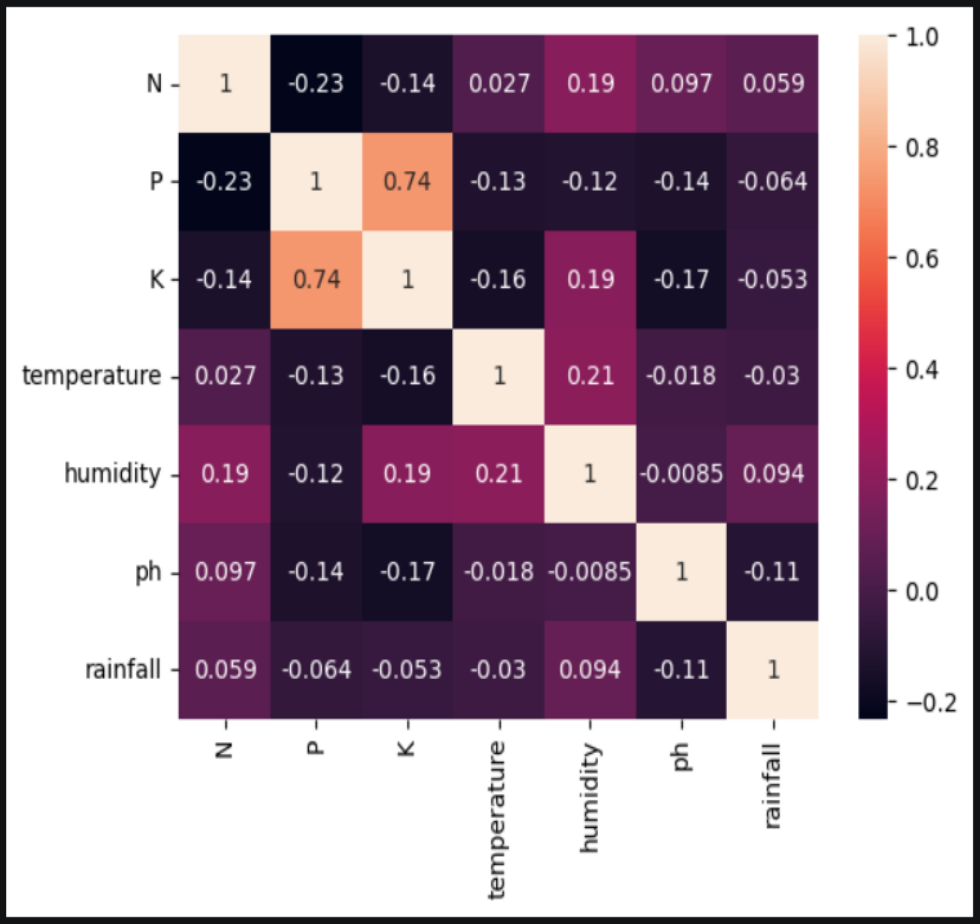



Crop Census
localhost/crops/index.php
CROP CENSUS
Home
Predictions
Recommendations
Sign in
Create account
Practical tools for better farming
Grow with
better insight.
Make confident decisions for your farm with data-powered crop, fertilizer, yield and rainfall tools.
EXPLORE RECOMMENDATIONS
See all tools
Thoughtful insights for every growing season
Smarter choices
Rooted in your farm data
Farm assistant
Make your next decision a better one.
Choose a tool to get useful guidance from the information you already know about your land.
RECOMMENDATION
Find the right crop
Match soil nutrients and local conditions to a crop recommendation.
Get started
RECOMMENDATION
Plan fertilizer use
Explore fertilizer guidance based on crop, soil and field conditions.
Get started
PREDICTION
Explore crop patterns
Look up crops grown by location and season.
Get started
PREDICTION
Estimate crop yield
Get a yield estimate using your crop, area and growing season.
Get started
PREDICTION
Check rainfall outlook
Review a regional rainfall estimate for your selected month.
Get started
MADE FOR THE FIELD
Useful, focused guidance
Simple tools designed to help turn farm details into a clearer next step.
How it works
Farm assistant

```
sns.heatmap(s,annot = True)
```

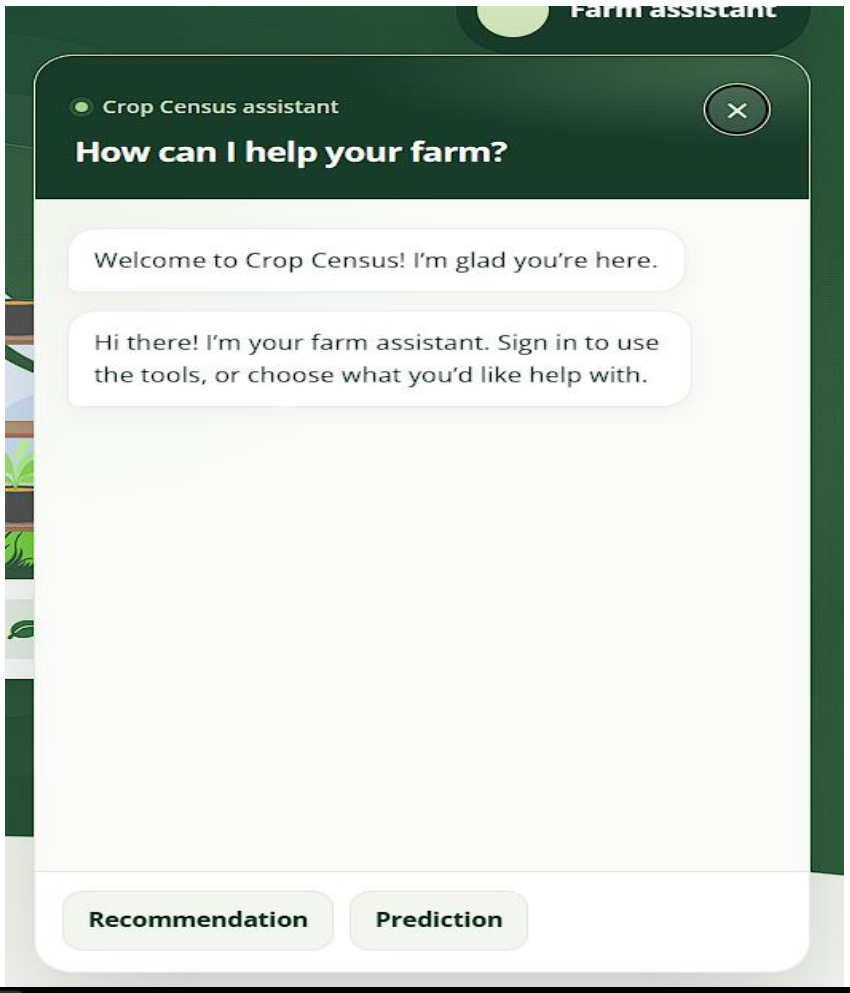
14]

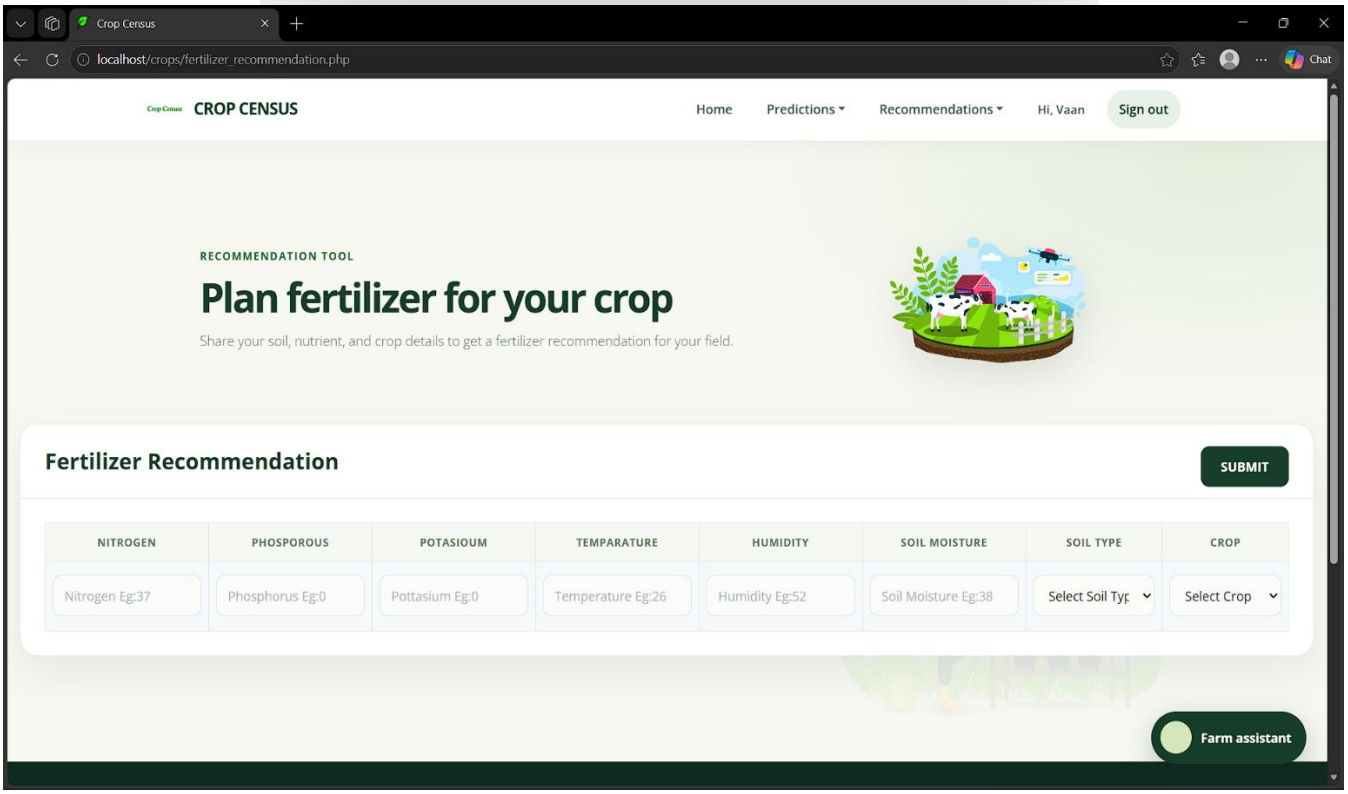
```
<AxesSubplot:>
```

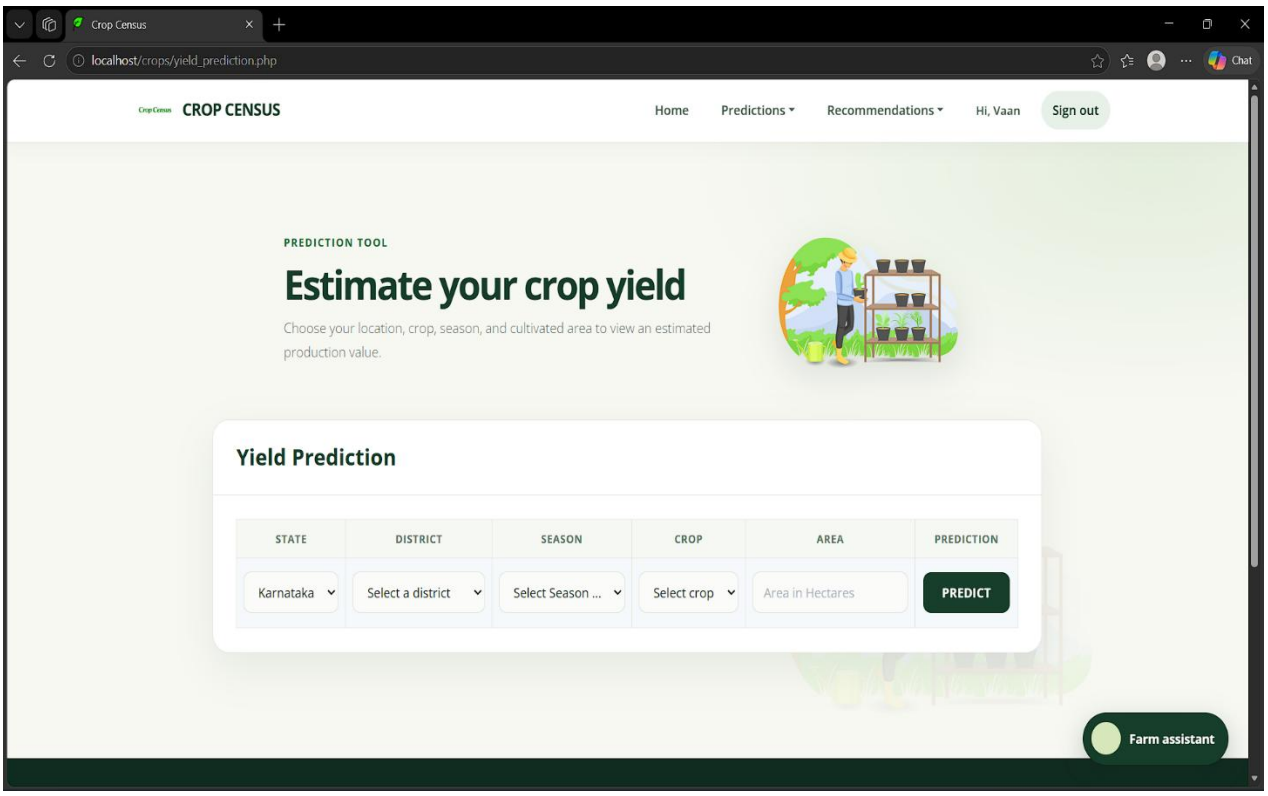


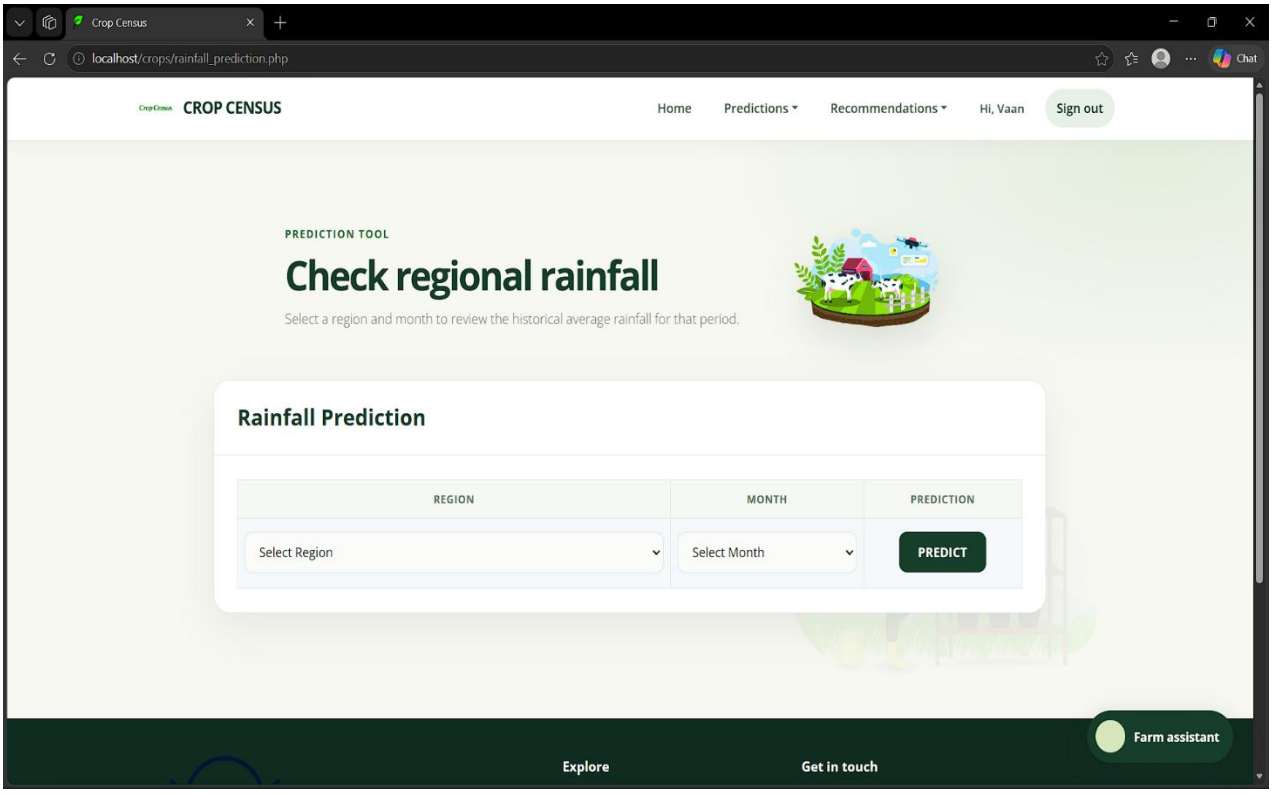

```
features = df[['N','P','K','temperature','humidity','ph','rainfall']]
target = df['label']
```

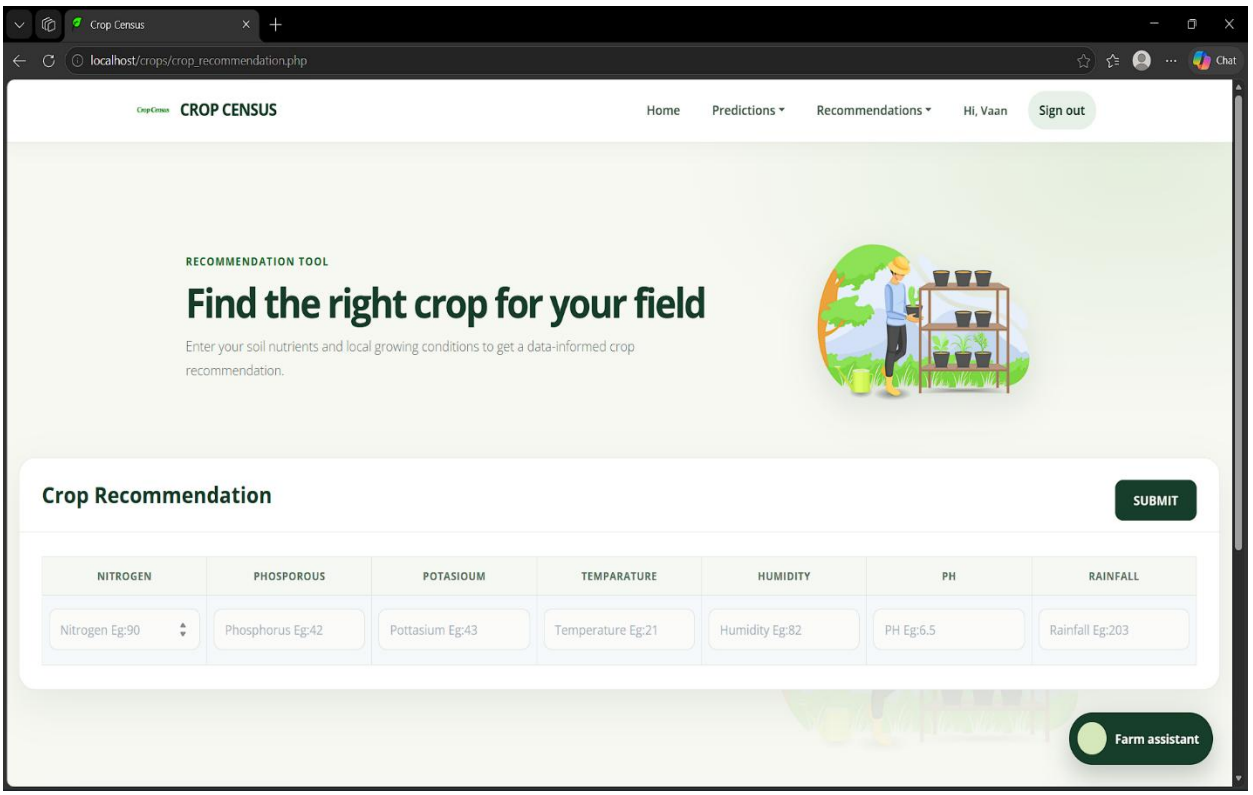
15]

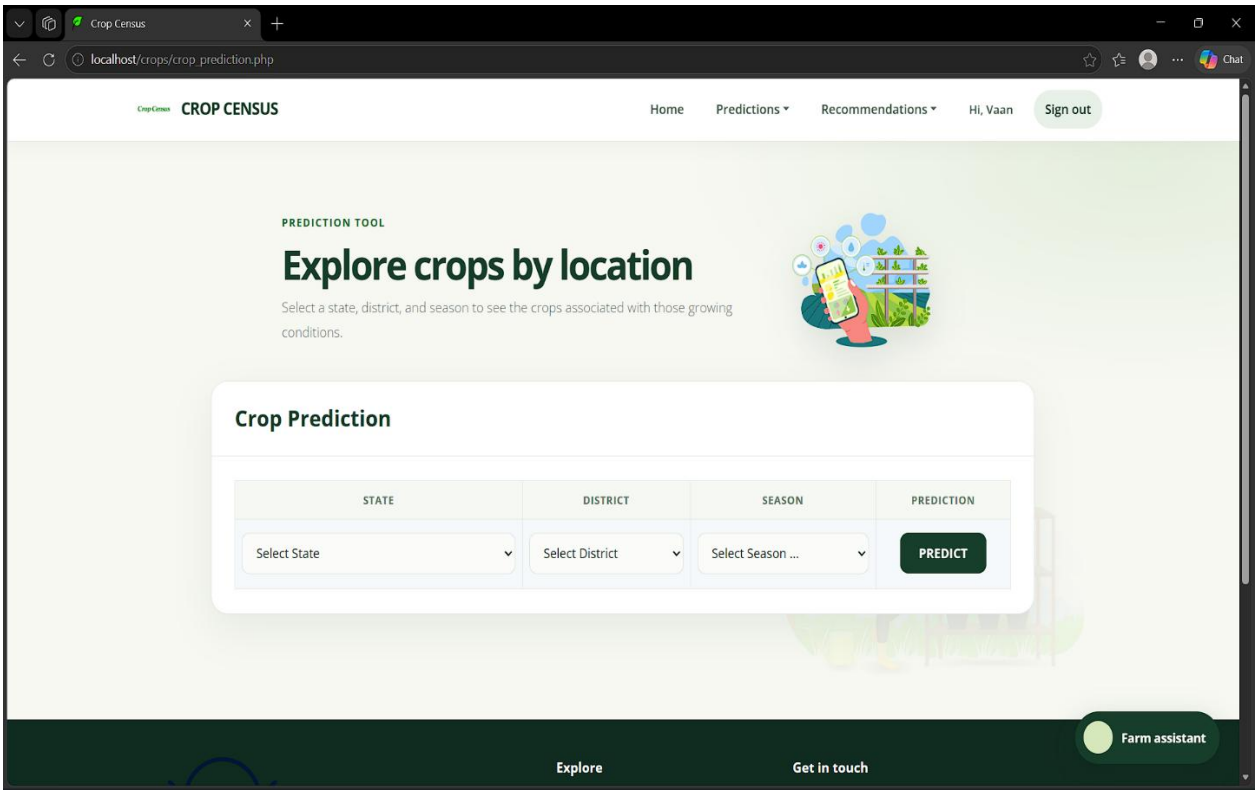

Crop Census assistant
How can I help your farm?
Welcome to Crop Census! I'm glad you're here.
Hi there! I'm your farm assistant. Sign in to use the tools, or choose what you'd like help with.
Recommendation
Prediction

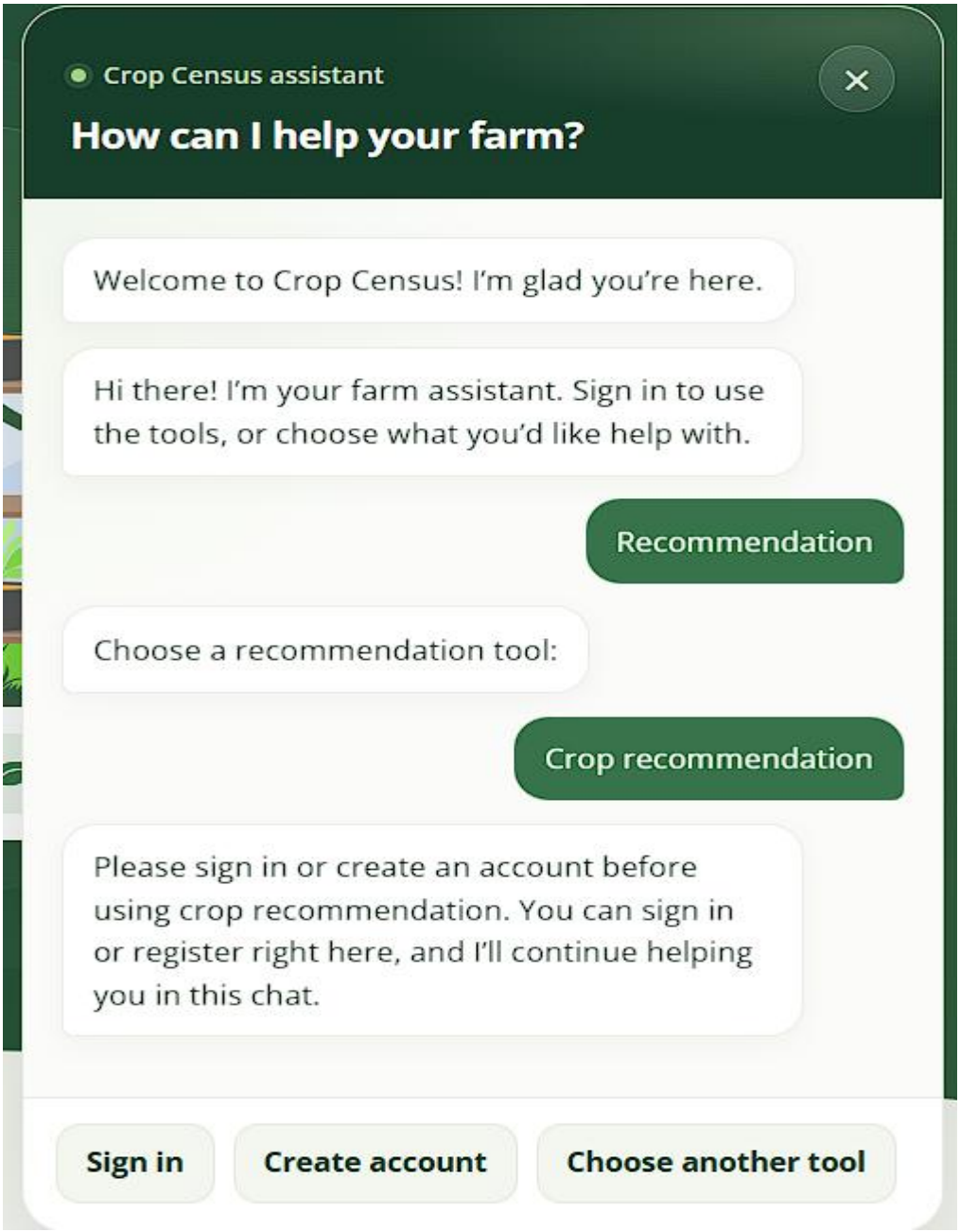

Crop Census
localhost/crops/fertilizer_recommendation.php
CROP CENSUS
Home
Predictions
Recommendations
Hi, Vaan
Sign out
RECOMMENDATION TOOL
Plan fertilizer for your crop
Share your soil, nutrient, and crop details to get a fertilizer recommendation for your field.
Fertilizer Recommendation
SUBMIT
NITROGEN
PHOSPOROUS
POTASIOUM
TEMPARATURE
HUMIDITY
SOIL MOISTURE
SOIL TYPE
CROP
Nitrogen Eg:37
Phosphorus Eg:0
Pottasium Eg:0
Temperature Eg:26
Humidity Eg:52
Soil Moisture Eg:38
Select Soil Typ
Select Crop
Farm assistant

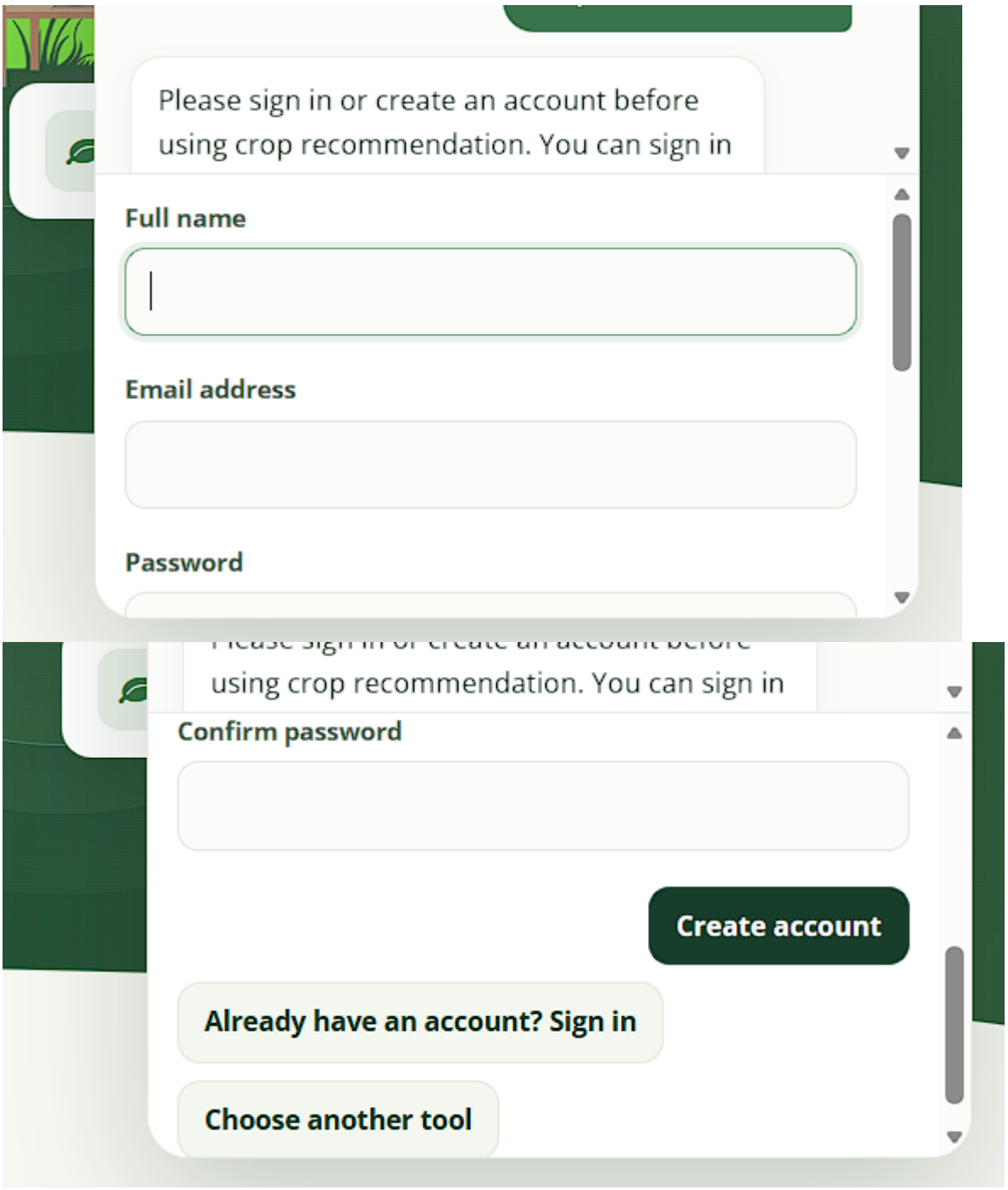

Crop Census
localhost/crops/yield_prediction.php
Chat
CROP CENSUS
Home
Predictions
Recommendations
Hi, Vaan
Sign out
PREDICTION TOOL
Estimate your crop yield
Choose your location, crop, season, and cultivated area to view an estimated production value.
Yield Prediction
STATE
DISTRICT
SEASON
CROP
AREA
PREDICTION
Karnataka
Select a district
Select Season ...
Select crop
Area in Hectares
PREDICT
Farm assistant

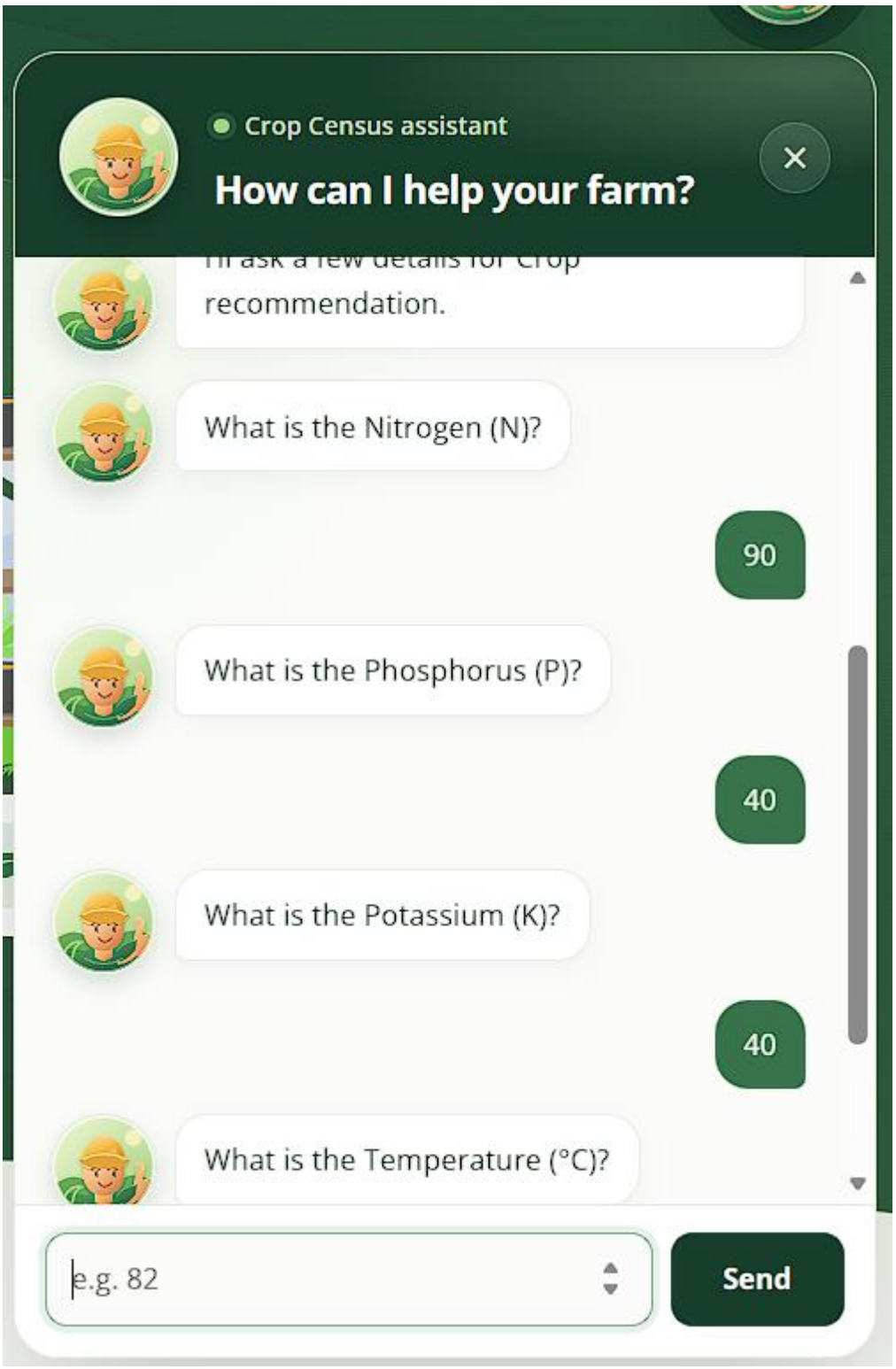

Crop Census
localhost/crops/rainfall_prediction.php
Chat
CROP CENSUS
Home
Predictions
Recommendations
Hi, Vaan
Sign out
PREDICTION TOOL
Check regional rainfall
Select a region and month to review the historical average rainfall for that period.
Rainfall Prediction
REGION
MONTH
PREDICTION
Select Region
Select Month
PREDICT
Farm assistant
Explore
Get in touch

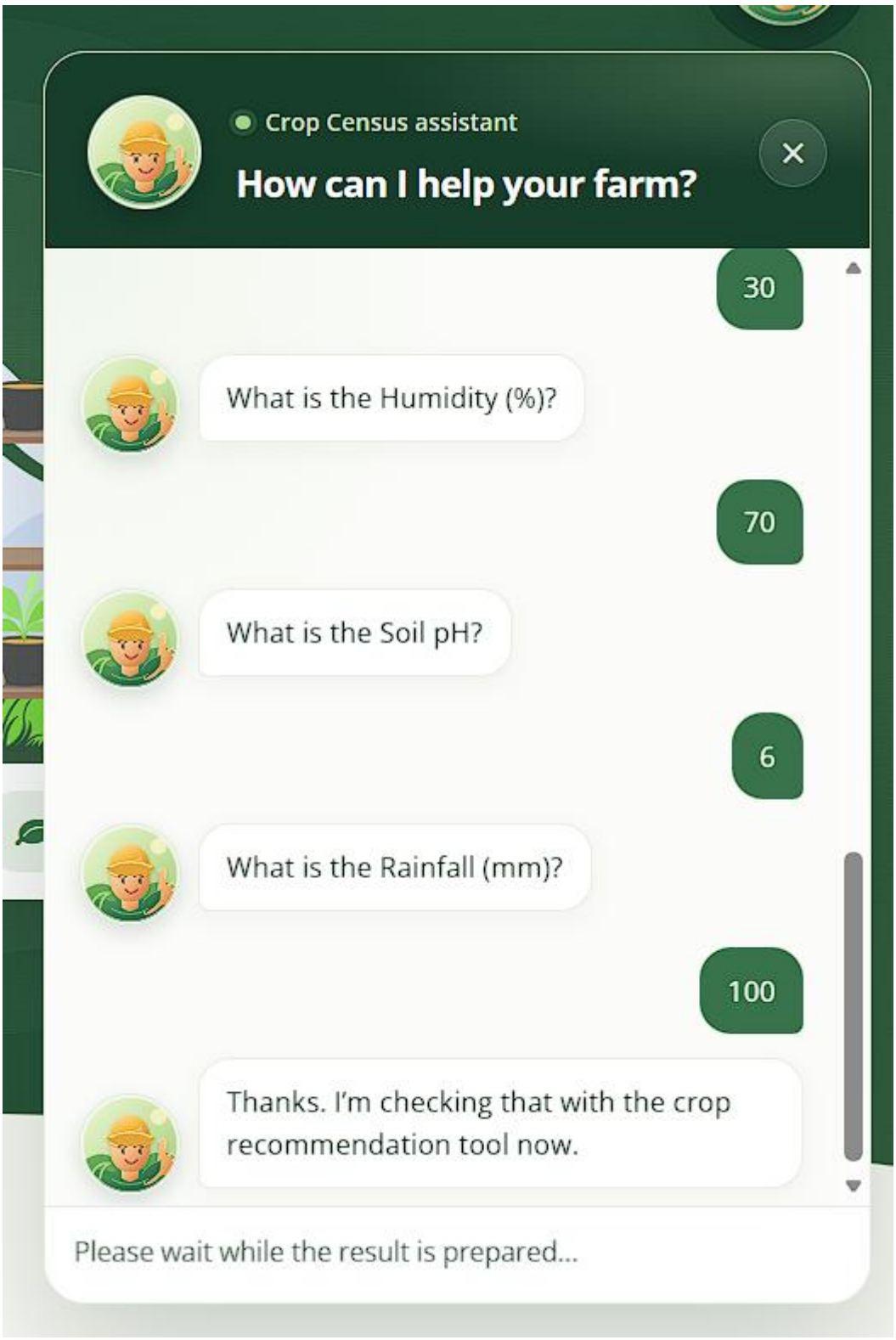

Crop Census
localhost/crops/crop_recommendation.php
CROP CENSUS
Home
Predictions
Recommendations
Hi, Vaan
Sign out
RECOMMENDATION TOOL
Find the right crop for your field
Enter your soil nutrients and local growing conditions to get a data-informed crop recommendation.
Crop Recommendation
SUBMIT
NITROGEN
PHOSPOROUS
POTASIOUM
TEMPARATURE
HUMIDITY
PH
RAINFALL
Nitrogen Eg:90
Phosphorus Eg:42
Pottasium Eg:43
Temperature Eg:21
Humidity Eg:82
PH Eg:6.5
Rainfall Eg:203
Farm assistant

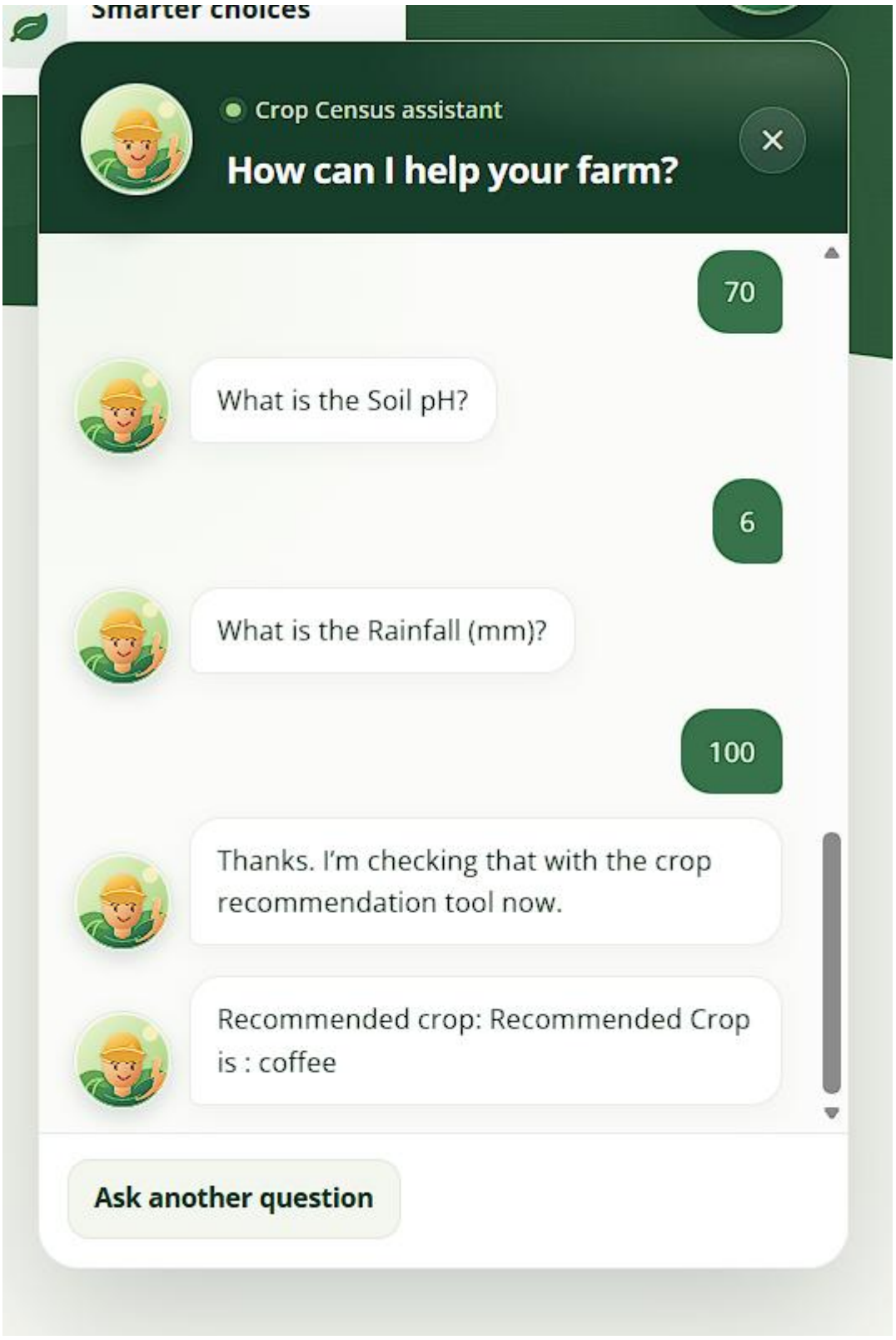

Crop Census
localhost/crops/crop_prediction.php
CROP CENSUS
Home
Predictions
Recommendations
Hi, Vaan
Sign out
PREDICTION TOOL
Explore crops by location
Select a state, district, and season to see the crops associated with those growing conditions.
Crop Prediction
STATE
DISTRICT
SEASON
PREDICTION
Select State
Select District
Select Season ...
PREDICT
Farm assistant
Explore
Get in touch

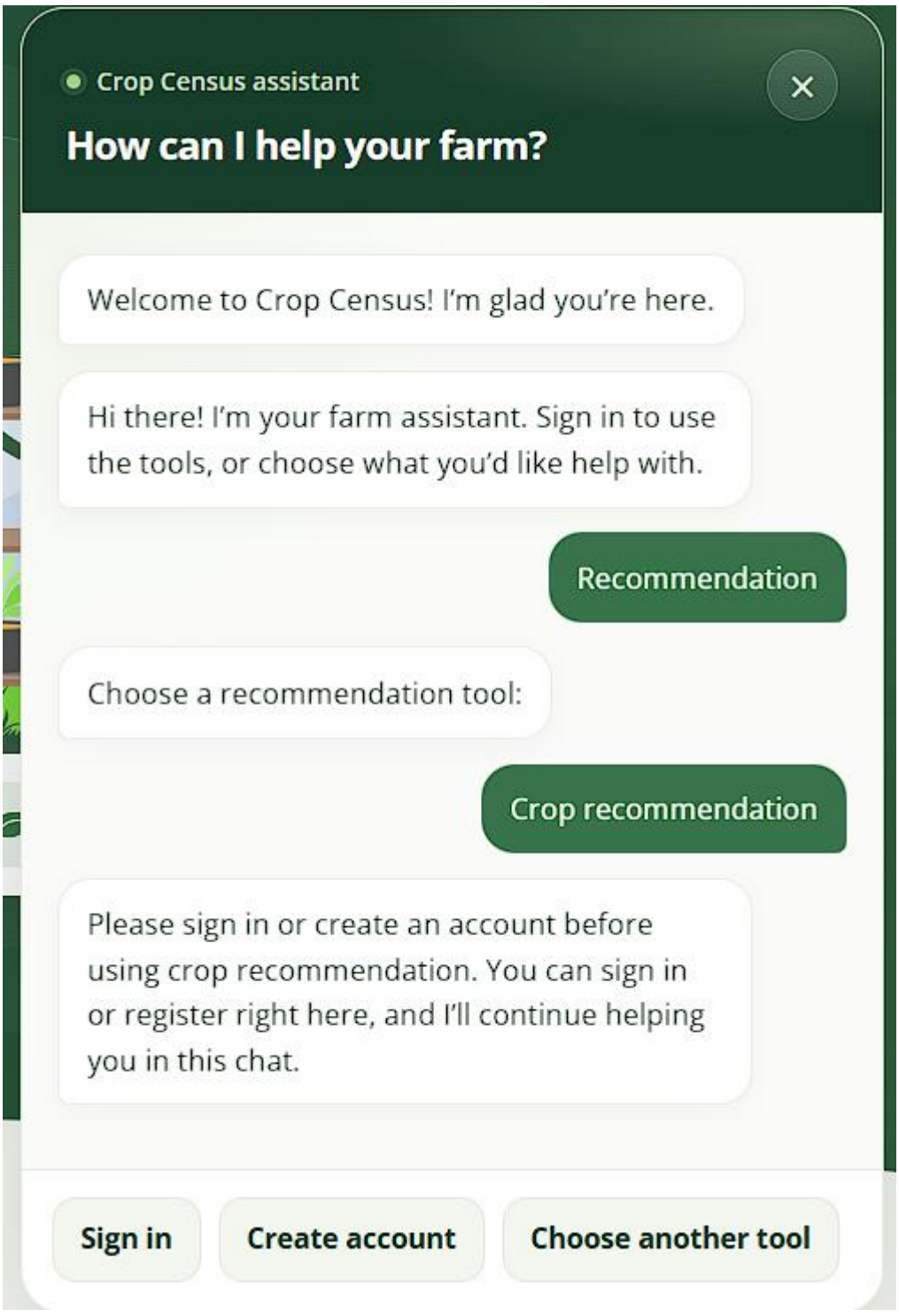

Crop Census assistant
How can I help your farm?
Welcome to Crop Census! I'm glad you're here.
Hi there! I'm your farm assistant. Sign in to use the tools, or choose what you'd like help with.
Recommendation
Choose a recommendation tool:
Crop recommendation
Please sign in or create an account before using crop recommendation. You can sign in or register right here, and I'll continue helping you in this chat.
Sign in
Create account
Choose another tool

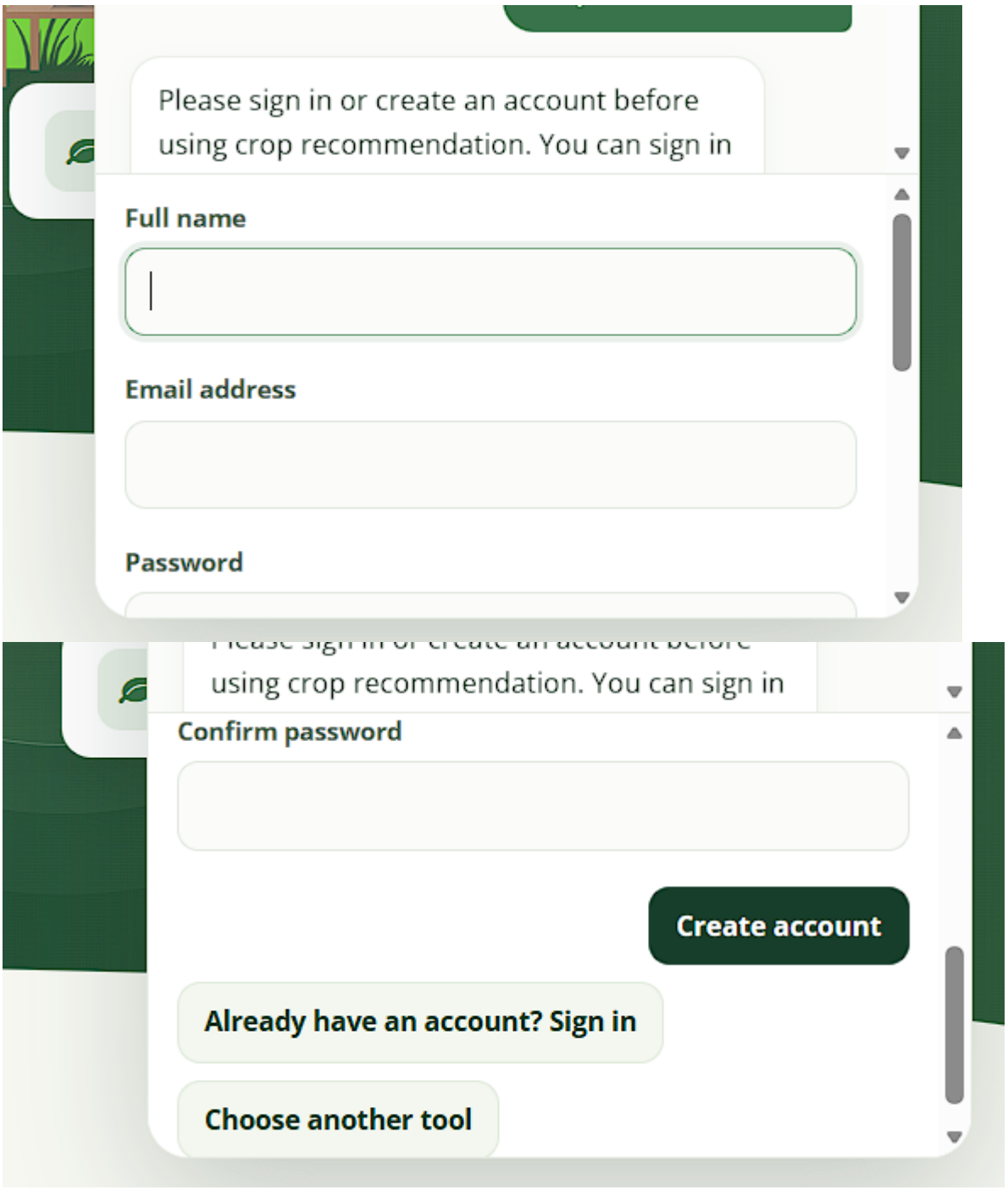

Please sign in or create an account before using crop recommendation. You can sign in
Full name
Email address
Password
using crop recommendation. You can sign in
Confirm password
Create account
Already have an account? Sign in
Choose another tool


Crop Census assistant
How can I help your farm?
recommendation.
What is the Nitrogen (N)?
90
What is the Phosphorus (P)?
40
What is the Potassium (K)?
40
What is the Temperature (°C)?
e.g. 82
Send


Crop Census assistant
How can I help your farm?
30
What is the Humidity (%)?
70
What is the Soil pH?
6
What is the Rainfall (mm)?
100
Thanks. I'm checking that with the crop recommendation tool now.
Please wait while the result is prepared...


Smarter choices
Crop Census assistant
How can I help your farm?
70
What is the Soil pH?
6
What is the Rainfall (mm)?
100
Thanks. I'm checking that with the crop recommendation tool now.
Recommended crop: Recommended Crop is : coffee
Ask another question


Crop Census assistant
How can I help your farm?
Welcome to Crop Census! I'm glad you're here.
Hi there! I'm your farm assistant. Sign in to use the tools, or choose what you'd like help with.
Recommendation
Choose a recommendation tool:
Crop recommendation
Please sign in or create an account before using crop recommendation. You can sign in or register right here, and I'll continue helping you in this chat.
Sign in
Create account
Choose another tool


Please sign in or create an account before using crop recommendation. You can sign in
Full name
Email address
Password
using crop recommendation. You can sign in
Confirm password
Create account
Already have an account? Sign in
Choose another tool

```
Decision Tree's accuracy is 90.0
              precision    recall  f1-score   support

       apple       1.00      1.00      1.00        13
      banana       1.00      1.00      1.00        17
   blackgram       0.59      1.00      0.74        16
    chickpea       1.00      1.00      1.00        21
     coconut       0.91      1.00      0.95        21
      coffee       1.00      1.00      1.00        22
      cotton       1.00      1.00      1.00        20
      grapes       1.00      1.00      1.00        18
        jute       0.74      0.93      0.83        28
 kidneybeans       0.00      0.00      0.00        14
      lentil       0.68      1.00      0.81        23
       maize       1.00      1.00      1.00        21
       mango       1.00      1.00      1.00        26
   mothbeans       0.00      0.00      0.00        19
    mungbean       1.00      1.00      1.00        24
   muskmelon       1.00      1.00      1.00        23
      orange       1.00      1.00      1.00        29
      papaya       1.00      0.84      0.91        19
  pigeonpeas       0.62      1.00      0.77        18
 pomegranate       1.00      1.00      1.00        17
        rice       1.00      0.62      0.77        16
  watermelon       1.00      1.00      1.00        15
...
    accuracy                           0.90       440
   macro avg       0.84      0.88      0.85       440
weighted avg       0.86      0.90      0.87       440

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

```
score = cross_val_score(DecisionTree,features,target,cv = 5)
```

```
df.shape
```

```
(2200, 8)
```

```
df.describe()
```

	N	P	K	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

```
df['label'].unique()
```

```
array(['rice', 'maize', 'chickpea', 'kidneybeans', 'pigeonpeas',
       'mothbeans', 'mungbean', 'blackgram', 'lentil', 'pomegranate',
       'banana', 'mango', 'grapes', 'watermelon', 'muskmelon', 'apple',
       'orange', 'papaya', 'coconut', 'cotton', 'jute', 'coffee'],
      dtype=object)
```

```
df.columns
```

```
Index(['N', 'P', 'K', 'temperature', 'humidity', 'ph', 'rainfall', 'label'], dtype='object')
```

```
df['label'].value_counts()
```

```
rice           100
maize          100
jute           100
cotton         100
coconut        100
papaya         100
orange         100
apple          100
muskmelon      100
watermelon     100
grapes         100
mango          100
banana         100
pomegranate    100
lentil         100
blackgram      100
mungbean       100
mothbeans      100
pigeonpeas     100
kidneybeans    100
chickpea       100
coffee         100
Name: label, dtype: int64
```

```
[12]
df.dtypes
```

```
N               int64
P               int64
K               int64
temperature   float64
humidity      float64
ph            float64
rainfall      float64
label          object
dtype: object
```

```
[13]
s = df.corr()
print(s)
```

```
                    N         P         K  temperature  humidity        ph  \
N            1.000000 -0.231460 -0.140512     0.026504  0.190688  0.096683
P           -0.231460  1.000000  0.736232    -0.127541 -0.118734 -0.138019
K           -0.140512  0.736232  1.000000    -0.160387  0.190859 -0.169503
temperature  0.026504 -0.127541 -0.160387     1.000000  0.205320 -0.017795
humidity     0.190688 -0.118734  0.190859     0.205320  1.000000 -0.008483
ph           0.096683 -0.138019 -0.169503    -0.017795 -0.008483  1.000000
rainfall     0.059020 -0.063839 -0.053461    -0.030084  0.094423 -0.109069

              rainfall
N             0.059020
P            -0.063839
K            -0.053461
temperature  -0.030084
humidity      0.094423
ph           -0.109069
rainfall      1.000000
```

```
df.tail()
```

	N	P	K	temperature	humidity	ph	rainfall	label
2195	107	34	32	26.774637	66.413269	6.780064	177.774507	coffee
2196	99	15	27	27.417112	56.636362	6.086922	127.924610	coffee
2197	118	33	30	24.131797	67.225123	6.362608	173.322839	coffee
2198	117	32	34	26.272418	52.127394	6.758793	127.175293	coffee
2199	104	18	30	23.603016	60.396475	6.779833	140.937041	coffee

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 2200 entries, 0 to 2199
Data columns (total 8 columns):
 #   Column       Non-Null Count  Dtype  
---  ------       --------------  -----  
 0   N            2200 non-null   int64  
 1   P            2200 non-null   int64  
 2   K            2200 non-null   int64  
 3   temperature  2200 non-null   float64
 4   humidity     2200 non-null   float64
 5   ph           2200 non-null   float64
 6   rainfall     2200 non-null   float64
 7   label        2200 non-null   object 
dtypes: float64(4), int64(3), object(1)
memory usage: 137.6+ KB
```

CHAPTER 8: GANTT CHART

The chart below shows an illustrative eight-week project schedule for developing and testing Crop Census, including the login-protected tools, visual design, and guided chatbot.

Project activity	W1	W2	W3	W4	W5	W6	W7	W8
Requirements gathering and analysis	■	■						
System design and diagrams		■	■					
Dataset preparation and model selection		■	■	■				
Database and user authentication			■	■				
Recommendation and prediction tools				■	■	■		
Website interface and agriculture illustrations				■	■	■		
Chatbot and in-chat sign-in/registration					■	■	■	
Integration and system testing						■	■	
Documentation and final review							■	■

Legend: ■ Activity in progress. The schedule is a sample; replace the week numbers with actual project dates if available.

CHAPTER 9 & 10: CONCLUSION AND FUTURE ENHANCEMENT

9 Conclusion

Crop Census brings several agricultural tools together in one web application. Farmers can use it to receive crop and fertilizer recommendations, explore crop patterns by location and season, estimate crop production, and review historical rainfall averages. User registration and sign-in protect access to the tools, while the guided chat assistant helps users choose a feature, provide its inputs, and view its result.

The application combines a interface, MySQL user accounts, Python scripts, and local agricultural datasets. Its current outputs should be understood as estimates based on those datasets and models: in particular, the rainfall tool calculates a historical monthly average, not a live forecast. Crop Census is intended to support planning, not replace local agricultural expertise.

10 Future Enhancements

Possible future improvements include:

-Improve prediction quality: Evaluate models using appropriate test data and metrics, compare algorithms, and document each model's limitations.

-Update agricultural data: Track dataset sources, collection dates, geographic coverage, and units; refresh datasets as reliable new information becomes available.

-Add live weather information: Integrate a trusted weather service to provide current conditions and forecasts alongside historical rainfall averages.

-Expand regional coverage: Add more locations and verify that each region, district, crop, and season is supported consistently across forms and models.

-Strengthen input validation: Check value ranges and units, handle unsupported selections clearly, and provide actionable errors if a model cannot return a result.

-Improve the chat assistant: Add multilingual support, clearer explanations of required measurements, and an option to review or correct answers before submitting them.

-Support responsible deployment: Add automated tests, monitoring, secure configuration guidance, and regular dependency and security reviews.

-Provide decision context: Explain what inputs a tool uses and present results with appropriate caveats so users can make informed decisions.

CHAPTER 11: BIBLIOGRAPHY

Scikit-learn Developers. Scikit-learn User Guide.

Documentation for classification, regression, decision trees, and ensemble methods.

Available at: https://scikit-learn.org/stable/user_guide.html.

pandas Development Team. pandas Documentation.

Documentation for reading and working with tabular datasets.

Available at: https://pandas.pydata.org/docs/.

HTML/PHP Documentation Group. HTML/PHP Manual.

Documentation for HTML/PHP language features, sessions, and database access.

Available at: https://www.php.net/manual/.

Oracle. MySQL Reference Manual.

Documentation for MySQL database concepts and SQL.

Available at: https://dev.mysql.com/doc/.